

# Einführung in R

Dr.-Ing. Grigory Devadze

# Was ist R?

- **Sprache:** R ist eine Programmiersprache für Statistik, Datenanalyse und Visualisierung, aus der akademischen Statistik entstanden, heute auch in Wirtschaft und Verwaltung Standard.
- **Ökosystem:** Im Alltag prägen einige Pakete den Kurs: `dplyr`, `tidyr`, `readr`, `ggplot2`, `readxl`, `haven`, `DBI`, `quarto`.
- **Arbeitsstil:** Typisch ist der Weg Konsole → Skript → Projekt → Report, nicht eine Kette klick-orientierter Einzelschritte.
- **Zielgruppen:** Besonders stark in Statistik, Biostatistik, Ökonometrie, Marktforschung, Ausbildung und reproduzierbarem Reporting.

# Was Sie mitnehmen

- Daten aus CSV, Excel, PDF und Datenbank sicher einlesen und mit dem tidyverse aufbereiten
- R-Skripte und -Projekte wartbar strukturieren, Fehler systematisch beheben
- Rechenprozesse als nachvollziehbaren R-Workflow: Zeitreihen aktualisieren, Gliederungsänderungen abbilden, mehrere Datenstände verknüpfen
- Zwischen- und Teilergebnisse als Tabelle oder Grafik in Datei oder Ausdruck ausgeben
- Prozesse dokumentieren und eine einfache Shiny-App bauen

# Teil I: RStudio, Projekte und Datenhaltung in R

# Von Excel zu R: was sich ändert

In Excel stehen Daten, Formeln und Ergebnis in denselben Zellen. In R trennen Sie diese drei Dinge: die Daten liegen als Datei, die Rechenschritte stehen als Skript, das Ergebnis entsteht beim Ausführen neu.

- Jeder Rechenschritt ist sichtbar und wiederholbar
- Eine Aktualisierung ist ein erneuter Lauf desselben Skripts, kein Nachziehen von Zellbezügen
- Der Weg vom Rohwert zur Tabelle ist nachvollziehbar und prüfbar

## Praxis

Das ist der Kern skriptbasierten Arbeitens: Sie beschreiben den Rechenweg einmal als Skript. Bei neuen Daten übergeben Sie nur den aktualisierten Datenstand und lassen denselben Code erneut laufen.

# Erste Schritte: R als Rechner

In der Console rechnet R sofort. Tippen Sie einen Ausdruck und führen Sie ihn aus:

```
3 + 4  
165000 * 1.012  
(120000 + 38000) / 2
```

R gibt das Ergebnis direkt zurück. Damit lässt sich jede Formel zunächst wie in einer Tabellenzelle ausprobieren, bevor sie ins Skript wandert.

# Zuweisung mit dem Pfeil

Ergebnisse merken Sie sich in **Objekten**. Die Zuweisung schreiben Sie mit `<-` (Kürzel `Alt` + `Minus`):

```
bws_be <- 165000  
quartale <- 4  
jahressumme <- bws_be * quartale  
jahressumme
```

- Links steht der Name, rechts der Wert
- Das Objekt erscheint danach im Environment
- Der Name darf Buchstaben, Punkt und Unterstrich enthalten, beginnt aber nicht mit einer Ziffer

## Stolperfalle

R unterscheidet Groß- und Kleinschreibung: `Wert` und `wert` sind zwei verschiedene Objekte.

# Funktionen aufrufen und Hilfe holen

Eine Funktion erkennen Sie an den runden Klammern. In den Klammern stehen die Argumente:

```
sum(165000, 38000, 28000)  
round(165123.47, 1)  
mean(c(96, 101, 100, 103))
```

Was eine Funktion tut und welche Argumente sie kennt, zeigt die Hilfe:

```
?round  
?mean
```

Die Hilfe öffnet sich rechts unten im Help-Pane. Jeder Hilfetext endet mit Beispielen unter „Examples“.

# Datentypen in R

Jeder Wert hat einen Typ. Die wichtigsten:

| Typ                    | Bedeutung                   | Beispiel                                      |
|------------------------|-----------------------------|-----------------------------------------------|
| <code>numeric</code>   | Zahlen                      | <code>165000</code> , <code>1.012</code>      |
| <code>character</code> | Text                        | <code>"BE"</code> , <code>"Baugewerbe"</code> |
| <code>logical</code>   | Wahrheitswert               | <code>TRUE</code> , <code>FALSE</code>        |
| <code>Date</code>      | Datum                       | <code>as.Date("2024-12-31")</code>            |
| <code>factor</code>    | Kategorie mit festen Stufen | Wirtschaftsbereich                            |

Den Typ eines Objekts prüfen Sie mit `class(x)` .

# Fehlende Werte: NA

Ein fehlender Wert heißt in R `NA` (not available). Das ist kein Text und keine Null, sondern „Wert unbekannt“.

```
werte <- c(165000, NA, 28000)
sum(werte)                # ergibt NA
sum(werte, na.rm = TRUE) # ignoriert NA
is.na(werte)              # TRUE an der Lücke
```

## Stolperfalle

Eine leere Excel-Zelle wird beim Einlesen oft zu `NA`. Rechnen Sie nicht blind weiter: prüfen Sie mit `is.na()` und entscheiden Sie bewusst über `na.rm = TRUE`.

# Faktoren und Datumswerte

**Faktor:** eine Textspalte mit fester Menge an Ausprägungen und definierter Reihenfolge.

```
bereich <- factor(c("BE", "F", "GI"),  
                 levels = c("A", "BE", "F", "GI"))  
levels(bereich)
```

**Datum:** ein eigener Typ, mit dem Sie rechnen und sortieren können.

```
stichtag <- as.Date("2024-12-31")  
paste0(format(stichtag, "%Y"), "-", quarters(stichtag)) # "2024-Q4"
```

`quarters()` ist Basis-R und liefert das zum Monat passende Quartal ( `"Q1"` bis `"Q4"` ). So leiten Sie das Quartal aus dem Datum ab, statt es fest zu verdrahten.

# Datenstrukturen: Vektor, data.frame, tibble, Liste

- **Vektor:** eine Folge gleichartiger Werte. Eine Tabellenspalte ist ein Vektor.
- **data.frame:** eine Tabelle aus gleich langen Spalten. Das Standardformat für Daten in R.
- **tibble:** die moderne data.frame-Variante aus dem tidyverse. Druckt übersichtlicher und macht weniger stillschweigende Umwandlungen.
- **Liste:** ein Behälter für beliebige, auch unterschiedliche Objekte (etwa mehrere Tabellen zugleich).

```
wert <- c(165000, 35000, 120000)
tab <- tibble::tibble(
  bereich_code = c("BE", "F", "GI"),
  wert        = wert
)
```

# Prinzip: Spalte = Variable, Zeile = Beobachtung

Eine saubere Datentabelle (tidy data) folgt einer Regel:

- Jede **Spalte** ist genau eine **Variable** (jahr, quartal, bereich\_code, wert)
- Jede **Zeile** ist genau eine **Beobachtung** (ein Wert für einen Bereich in einem Quartal)
- Jede Zelle enthält genau einen Wert

Dieses Format heißt **long** (lang). Im Gegensatz dazu hat das **wide**-Format (breit) eine Spalte je Quartal, wie man es aus Excel kennt.

## Praxis

Der Datensatz liegt im long-Format vor. Die Excel-Datei im selben Pack liegt im wide-Format. In Teil II lernen Sie, zwischen beiden umzuformen.

# Datenformate auf der Platte

| Format  | Wofür                                           | Lesen mit                          |
|---------|-------------------------------------------------|------------------------------------|
| CSV     | universeller Austausch, eine Tabelle als Text   | <code>readr::read_csv()</code>     |
| Excel   | mehrere Blätter, Behördenformat mit Titelzeilen | <code>readxl::read_excel()</code>  |
| RDS     | ein R-Objekt 1:1 sichern, schnell und typtreu   | <code>readRDS()</code>             |
| RData   | mehrere Objekte in einer Datei                  | <code>load()</code>                |
| Parquet | große Datenmengen, spaltenweise, sehr kompakt   | <code>arrow::read_parquet()</code> |

Für Zwischenergebnisse ist **RDS** meist die beste Wahl: es bewahrt Typen wie Faktor und Datum exakt, anders als CSV.

# R-Projekte (.Rproj)

Ein **R-Projekt** bündelt alles, was zu einer Aufgabe gehört, in einem Ordner. Beim Öffnen der `.Rproj`-Datei setzt RStudio das **Arbeitsverzeichnis** automatisch auf diesen Ordner.

- Alle Pfade bleiben relativ und funktionieren auch auf dem Rechner der Kollegin
- Kein `setwd("C:/Users/...")` mehr, das nur bei Ihnen läuft
- `here::here("data", "lieferung_2024q4.csv")` baut Pfade stabil ab dem Projektordner

Empfohlene Ordnerstruktur:

```
projekt/  
  projekt.Rproj  
  data/      # Rohdaten, unverändert  
  R/         # Skripte  
  output/    # erzeugte Tabellen und Grafiken
```

## Stolperfalle

Schreiben Sie nie in `data/` zurück. Rohdaten bleiben unangetastet, alle Ergebnisse landen in `output/`.

# Aufbau eines guten R-Skripts

Ein lesbares Skript folgt immer demselben Muster:

```
# -----  
# Lieferung 2024Q4 aufbereiten  
# Autor: ...   Stand: 2024-12-15  
# -----  
  
# ---- Pakete ----  
library(readr)  
library(dplyr)  
  
# ---- Daten einlesen ----  
bws <- read_csv(here::here("data", "lieferung_2024q4.csv"))  
  
# ---- Aufbereitung ----  
# ...  
  
# ---- Ausgabe ----  
# ...
```

Erst der Kopf, dann die Pakete, dann die fachlichen Abschnitte: dieselbe Reihenfolge in jedem Skript.

# Aufbau eines guten R-Skripts: die Konventionen

Drei Gewohnheiten machen ein Skript lesbar und übergabefähig:

- **Kopfkommentar:** Zweck, Autor, Stand. So weiß die Kollegin sofort, was das Skript tut und wie aktuell es ist.
- `library()` -Block ganz oben: alle Pakete gebündelt an einer Stelle, nicht über das Skript verstreut.
- **Abschnitte mit `# ---- Titel ----`** : RStudio macht daraus eine navigierbare Gliederung (Sprungmarken im Editor).

# Pakete: installieren und laden

R bringt Grundfunktionen mit. Zusatzfunktionen kommen aus **Paketen**:

```
install.packages("tidyverse")  # einmal pro Rechner: herunterladen  
library(tidyverse)             # in jeder Sitzung: aktivieren
```

- `install.packages()` lädt das Paket aus dem Internet und legt es ab (einmalig)
- `library()` macht die Funktionen in der aktuellen Sitzung verfügbar (in jedem Skript erneut)

Das **tidyverse** ist eine Sammlung zusammenpassender Pakete für Datenarbeit: `readr` (einlesen), `dplyr` (transformieren), `tidyr` (umformen), `ggplot2` (visualisieren), `stringr`, `lubridate` und weitere. Wir nutzen es als roten Faden der Schulung.

# Teil II: Daten einlesen und mit dem tidyverse verarbeiten

# Was Sie in diesem Teil tun

- Eine CSV-Datei sauber einlesen und die Spaltentypen prüfen
- Die Pipe `%>%` als roten Faden durch jede Auswertung legen
- Die dplyr-Kernverben einzeln kennenlernen und kombinieren
- Aggregieren, neue Kennzahlen berechnen und das Format wechseln (long/wide)

## Praxis

Datensatz: `lieferung_2024q4.csv` , die Bruttowertschöpfung (BWS) in jeweiligen Preisen je Quartal und Wirtschaftsbereich. Spalten: `jahr` , `quartal` , `bereich_code` , `bereich_name` , `wert` (Mio. EUR).

# CSV einlesen mit readr

`readr::read_csv()` ist die tidyverse-Variante zum Einlesen. Sie liefert ein `tibble`, meldet die erkannten Spaltentypen und ist schneller als base R.

```
library(tidyverse)

daten <- read_csv("data/lieferung_2024q4.csv")
# Rows: 280 Columns: 5
# Column specification
#   jahr, quartal      <dbl> (numerisch)
#   bereich_code, ...  <chr>  (Text)
#   wert              <dbl> (numerisch)
```

Der Datenpfad ist relativ zum geöffneten RStudio-Projekt (also `data/...`).

# read\_csv() gegenüber base read.csv()

## readr::read\_csv()

- Ergebnis: `tibble`
- Text bleibt Text (kein Faktor-Zwang)
- Typ-Erkennung wird gemeldet
- Spaltennamen unverändert

## utils::read.csv()

- Ergebnis: `data.frame`
- früher `stringsAsFactors = TRUE`
- still, ohne Typ-Report
- `make.names()` verändert Namen

Für reproduzierbare Auswertungen ist die explizite, sichtbare Typ-Erkennung von `read_csv()` der ehrlichere Weg.

# Spaltentypen explizit festlegen

Verlassen Sie sich bei produktiven Prozessen nicht allein auf die automatische Erkennung. Geben Sie die Typen vor, dann fällt eine abweichende Eingabedatei sofort auf.

```
daten <- read_csv(  
  "data/lieferung_2024q4.csv",  
  col_types = cols(  
    jahr          = col_integer(),  
    quartal       = col_integer(),  
    bereich_code  = col_character(),  
    bereich_name  = col_character(),  
    wert          = col_double()  
  )  
)
```

Mit `spec(daten)` sehen Sie die geratene Spezifikation und können sie als Vorlage übernehmen.

# Dezimalkomma in deutschen Exporten

## Stolperfalle

Exporte aus deutschen Systemen schreiben Zahlen oft als `1.234,5` (Punkt als Tausender-, Komma als Dezimaltrennzeichen). `read_csv()` erwartet standardmäßig den Punkt als Dezimaltrennzeichen und liest solche Werte sonst als Text ein.

```
de <- read_csv2(  
  "data/lieferung_de.csv",  
  locale = locale(decimal_mark = ",", grouping_mark = ".")  
)
```

`read_csv2()` setzt Semikolon als Trenner und Komma als Dezimalzeichen voreingestellt; mit `locale()` stellen Sie beides bei Bedarf einzeln ein.

# Erster Blick auf die Daten

```
glimpse(daten)  # je Spalte: Typ + erste Werte, ideal bei vielen Spalten
head(daten)     # die ersten 6 Zeilen
summary(daten)  # Kennzahlen je Spalte (Min, Median, Max, ...)
View(daten)     # tabellarische Ansicht in RStudio
```

## Praxis

`glimpse()` zeigt sofort, ob `jahr` und `quartal` als Zahl und `wert` als `<dbl>` erkannt wurden: ein schneller Plausibilitätscheck für jede neue Eingabedatei.

# Die Pipe als roter Faden

Die tidyverse-Pipe `%>%` reicht das Ergebnis links als erstes Argument an die Funktion rechts weiter. So liest sich ein Ablauf von oben nach unten wie eine Folge von Arbeitsschritten.

```
# verschachtelt, schwer lesbar:  
arrange(filter(daten, jahr == 2024), desc(wert))  
  
# als Pipe, Schritt für Schritt:  
daten %>%  
  filter(jahr == 2024) %>%  
  arrange(desc(wert))
```

Lesen Sie `%>%` gedanklich als „und dann“. Alternativ steht seit R 4.1 die Base-R-Pipe `|>` zur Verfügung. In diesem Kurs verwenden wir einheitlich `%>%`; in den hier gezeigten einfachen Ketten sind beide Schreibweisen möglich.

# dplyr-Kernverben im Überblick

| Verb                     | Aufgabe                           |
|--------------------------|-----------------------------------|
| <code>filter()</code>    | Zeilen nach Bedingung auswählen   |
| <code>select()</code>    | Spalten auswählen oder umbenennen |
| <code>arrange()</code>   | Zeilen sortieren                  |
| <code>mutate()</code>    | neue Spalten berechnen            |
| <code>summarise()</code> | Zeilen zu Kennzahlen verdichten   |
| <code>group_by()</code>  | Operationen je Gruppe ausführen   |

Jedes Verb nimmt als erstes Argument einen Datensatz und gibt einen Datensatz zurück: ideal für die Pipe.

# filter() und select()

```
# Zeilen: nur das vierte Quartal 2024
daten %>%
  filter(jahr == 2024, quartal == 4)

# Spalten: Zeit, Bereich und Wert behalten
daten %>%
  select(jahr, quartal, bereich_code, wert)

# umbenennen direkt in select():
daten %>%
  select(jahr, quartal, code = bereich_code, wert)
```

`filter()` wählt Zeilen, `select()` wählt Spalten. Mehrere Bedingungen in `filter()` werden mit Komma als logisches UND verknüpft.

# arrange(): Zeilen sortieren

```
# aufsteigend nach Wert
daten %>% arrange(wert)

# absteigend: größte Bereiche zuerst
daten %>% arrange(desc(wert))

# mehrere Kriterien: erst Jahr, dann Wert absteigend
daten %>% arrange(jahr, desc(wert))
```

`arrange()` sortiert die Zeilen, ohne sie zu verändern. `desc()` dreht die Reihenfolge einer Spalte um; weitere Spalten dienen als Nebenkriterium bei Gleichstand.

# mutate() und summarise()

```
# mutate(): neue Spalte, alle Zeilen bleiben erhalten
daten %>% mutate(wert_mrd = wert / 1000)

# summarise(): viele Zeilen zu einer Kennzahl verdichten
daten %>%
  summarise(bws_total = sum(wert))
```

`mutate()` behält alle Zeilen und fügt Spalten hinzu; `summarise()` reduziert viele Zeilen auf eine Kennzahl. Diesen Gegensatz (rechnen je Zeile gegen verdichten) nutzen Sie auf den folgenden Folien gruppenweise.

# Filterung vertiefen

```
# Vergleichsoperatoren
daten %>% filter(wert > 50000, jahr >= 2022)

# mehrere Werte: %in% statt vieler ODER-Bedingungen
daten %>% filter(bereich_code %in% c("BE", "GI", "OQ"))

# Wertebereich: between() ist Kurzform für >= und <=
daten %>% filter(between(jahr, 2020, 2024))
```

## Stolperfalle

Auf Gleichheit prüfen Sie mit doppeltem `==`, nicht mit einfachem `=`. Ein einfaches `=` ist eine Zuweisung und führt in `filter()` zu einem Fehler.

# Aggregation: `group_by()` und `summarise()`

`group_by()` definiert die Gruppen, `summarise()` rechnet je Gruppe eine Kennzahl.

```
# BWS-Summe je Jahr
daten %>%
  group_by(jahr) %>%
  summarise(bws_total = sum(wert))

# Durchschnitt je Wirtschaftsbereich
daten %>%
  group_by(bereich_code, bereich_name) %>%
  summarise(mittel = mean(wert), .groups = "drop")
```

## Stolperfalle

Nach `summarise()` bleibt eine Gruppierungsebene aktiv. Setzen Sie `.groups = "drop"` oder ein abschließendes `ungroup()`, sonst rechnen spätere Schritte unbeabsichtigt je Gruppe weiter.

# Anteil je Gruppe mit mutate()

```
# Anteil jedes Bereichs am Quartalstotal
daten %>%
  group_by(jahr, quartal) %>%
  mutate(anteil = wert / sum(wert)) %>%
  ungroup()
```

`mutate()` mit `group_by()` rechnet je Gruppe, behält aber alle Zeilen: `sum(wert)` ist hier die Summe innerhalb des jeweiligen Quartals, nicht über den ganzen Datensatz. Das abschließende `ungroup()` hebt die Gruppierung wieder auf.

# Veränderung zum Vorquartal mit lag()

```
# Vorquartalsveränderung je Bereich
daten %>%
  arrange(bereich_code, jahr, quartal) %>%
  group_by(bereich_code) %>%
  mutate(
    vorquartal = lag(wert),
    veraenderung = wert / vorquartal - 1
  ) %>%
  ungroup()
```

`lag()` greift auf den vorherigen Wert in der sortierten Reihenfolge zu. Sortieren Sie vorher mit `arrange()`, sonst ist „vorher“ nicht eindeutig. `group_by(bereich_code)` sorgt dafür, dass `lag()` nicht über Bereichsgrenzen hinweg vergleicht.

# Zählen mit `count()` und `distinct()`

```
# wie viele Zeilen je Jahr?  
daten %>% count(jahr)  
  
# wie viele Zeilen je Bereich, absteigend sortiert  
daten %>% count(bereich_code, sort = TRUE)  
  
# welche Bereiche kommen überhaupt vor?  
daten %>% distinct(bereich_code, bereich_name)
```

`count()` ist die Kurzform für `group_by() %>% summarise(n = n())`. `distinct()` liefert die eindeutigen Kombinationen: ein schneller Test auf vollständige und doppelte Schlüssel.

# Format wechseln: pivot\_longer() und pivot\_wider()

## long (tidy)

eine Zeile je Beobachtung; so liegt `daten` vor  
und so rechnet dplyr am bequemsten.

## wide

eine Spalte je Merkmalsausprägung; gut für  
Tabellen und den Export nach Excel.

```
# long -> wide: Bereiche in Spalten, je Zeit eine Zeile
breit <- daten %>%
  pivot_wider(names_from = bereich_code, values_from = wert)

# wide -> long: zurück ins tidy-Format
lang <- breit %>%
  pivot_longer(
    cols = -c(jahr, quartal),
    names_to = "bereich_code",
    values_to = "wert"
  )
```

# Typische Aufbereitung als dplyr-Kette

## Praxis

Ein wiederkehrender Schritt: nach Quartal filtern, je Bereich aggregieren, Anteil am Total berechnen und absteigend sortieren. Eine durchgehende Kette ersetzt eine ganze Folge manueller Excel-Schritte und ist exakt reproduzierbar.

```
daten %>%  
  filter(jahr == 2024, quartal == 4) %>%  
  group_by(bereich_code, bereich_name) %>%  
  summarise(wert = sum(wert), .groups = "drop") %>%  
  mutate(anteil = wert / sum(wert)) %>%  
  arrange(desc(anteil))
```

# Teil III: Externe Datenquellen und Excel-Schnittstellen

# Worum es in diesem Teil geht

Im VGR-Bereich liegen Daten selten nur als saubere CSV vor: Excel-Dateien mit Titelzeilen, PDF-Berichte, eine Fachdatenbank wie EDO. R liest all das mit denselben Mitteln ein und schreibt Ergebnisse auch wieder zurück.

- Excel lesen und schreiben ( `readxl` , `openxlsx` )
- Universal-Import mit `rio`
- PDF-Tabellen mit `pdftools` parsen
- Datenbankzugriff über `DBI` (SQLite-Demo als EDO-Proxy)
- Wann R, wann Excel: ein reproduzierbarer Übergang statt Copy-Paste

## Praxis

Ziel ist, jede wiederkehrende Eingabe über denselben, dokumentierten Einleseweg zu führen: nachvollziehbar, wiederholbar, ohne manuelles Kopieren zwischen Dateien.

# Excel lesen mit readxl

`readxl::read_excel()` liest `.xls` und `.xlsx` ohne externe Abhängigkeiten. Die drei wichtigsten Argumente steuern, welcher Ausschnitt gelesen wird.

```
library(readxl)

# Erstes Blatt, alles ab Zeile 1
read_excel("data/lieferung_excel.xlsx")

# Bestimmtes Blatt per Name oder Nummer
read_excel("data/lieferung_excel.xlsx", sheet = "Daten")
read_excel("data/lieferung_excel.xlsx", sheet = 2)
```

- `sheet` : Blatt per Name (robust) oder Position
- `skip` : Anzahl Kopfzeilen, die übersprungen werden
- `range` : exakter Zellbereich, etwa `"A4:AD14"`

# Titelzeilen über der Tabelle

In unserer Demo-Datei stehen im Blatt `Daten` zwei Titelzeilen und eine Leerzeile über der eigentlichen Tabelle. Die Spaltenüberschriften stehen in Zeile 4, die Werte ab Zeile 5.

```
# Falsch: read_excel nimmt die Titelzeile als Spaltennamen
read_excel("data/lieferung_excel.xlsx", sheet = "Daten")

# Richtig: die ersten drei Zeilen überspringen
read_excel("data/lieferung_excel.xlsx", sheet = "Daten", skip = 3)
```

## Stolperfalle

Excel-Dateien tragen oft Titel, Stand und Einheit über der Tabelle. Ohne `skip` oder `range` landen diese Texte als kaputte Spaltennamen ( `...1` , `...2` ) in den Daten. Vor dem Einlesen kurz in Excel prüfen, in welcher Zeile die Tabelle wirklich beginnt.

# Genauen Bereich mit range lesen

`range` ist die präziseste Variante: Sie lesen exakt den Block, der die Tabelle enthält, und ignorieren alles darum herum (Titel, Fußnoten, Nebenrechnungen rechts daneben).

```
# Zellbereich in A1-Notation
read_excel("data/lieferung_excel.xlsx",
           sheet = "Daten",
           range = "A4:AD14")

# Bereich auf einem benannten Blatt
read_excel("data/lieferung_excel.xlsx",
           range = "Daten!A4:AD14")
```

`range` schlägt `skip`, wenn das Blatt unten oder rechts Störzeilen enthält. Bei wachsenden Tabellen ist `skip` flexibler, weil es die Zeilenzahl nicht festnagelt.

# Mehrere Tabellenblätter

Eine Quelldatei kann Metadaten und Daten auf getrennten Blättern führen. `excel_sheets()` listet alle Blattnamen, bevor Sie etwas einlesen.

```
library(readxl)

excel_sheets("data/lieferung_excel.xlsx")
#> [1] "Metadaten" "Daten"
```

So sehen Sie die Struktur einer fremden Datei, ohne sie in Excel öffnen zu müssen, und greifen anschließend gezielt auf das richtige Blatt zu.

# Über alle Blätter iterieren

Wenn jedes Blatt dieselbe Struktur hat (etwa ein Blatt je Quartal), lesen Sie alle in einem Schritt ein und binden sie zusammen.

`purrr::map()` liefert eine Liste, `set_names()` behält die Blattnamen als Herkunft.

```
library(readxl)
library(purrr)
library(dplyr)

pfad <- "data/lieferung_excel.xlsx"

blaetter <- excel_sheets(pfad)

alle <- blaetter %>%
  set_names() %>%
  map(\(bl) read_excel(pfad, sheet = bl, skip = 3)) %>%
  list_rbind(names_to = "blatt")
```

## Praxis

Typischer Fall: eine Arbeitsmappe mit einem Blatt je Monatsbericht. Statt jedes Blatt von Hand zu kopieren, liest die Schleife alle Blätter ein und vermerkt in der Spalte `blatt`, woher jede Zeile stammt.

`map(...)` %>% `list_rbind()` ist das aktuelle Muster fürs zeilenweise Binden. Das früher übliche `map_dfr()` macht dasselbe, gilt aber als superseded.

# Ergebnisse nach Excel schreiben mit openxlsx

Der Rückweg R nach Excel: `openxlsx` baut eine Arbeitsmappe im Code auf, schreibt Daten hinein und kann Spaltenbreiten, Kopfzeilen und Formate setzen, ohne dass Excel installiert sein muss.

```
library(openxlsx)

wb <- createWorkbook()
addWorksheet(wb, "Ergebnis")
writeData(wb, "Ergebnis", ergebnis_tabelle)

saveWorkbook(wb, "output/ergebnis.xlsx", overwrite = TRUE)
```

`createWorkbook()` legt die leere Mappe an, `addWorksheet()` ein Blatt, `writeData()` schreibt einen Data Frame inklusive Spaltennamen, `saveWorkbook()` speichert.

# Einfache Formatierung in openxlsx

Für eine vorzeigbare Ausgabe reichen wenige Stilangaben: fette Kopfzeile, Tausendertrennung, passende Spaltenbreite.

```
library(openxlsx)

wb <- createWorkbook()
addWorksheet(wb, "Ergebnis")
writeData(wb, "Ergebnis", ergebnis_tabelle)

kopf <- createStyle(textDecoration = "bold", fgFill = "#21303f",
                    fontColour = "#ffffff")
addStyle(wb, "Ergebnis", kopf, rows = 1,
         cols = seq_along(ergebnis_tabelle), gridExpand = TRUE)

setColWidths(wb, "Ergebnis", cols = seq_along(ergebnis_tabelle),
             widths = "auto")

saveWorkbook(wb, "output/ergebnis.xlsx", overwrite = TRUE)
```

## Stolperfalle

`overwrite = TRUE` ist nötig, wenn die Datei schon existiert. Ohne diese Angabe bricht `saveWorkbook()` ab, statt eine bestehende Datei versehentlich zu ersetzen.

# rio als Universal-Import und -Export

`rio` ist eine Schicht über vielen Einzelpaketen: `import()` und `export()` erkennen das Format an der Dateiendung. Praktisch für schnelle Konvertierungen und einheitliche Schreibweise im Skript.

```
library(rio)

# Format wird an der Endung erkannt
daten    <- import("data/lieferung_2024q4.csv")
aus_xl   <- import("data/lieferung_excel.xlsx", which = "Daten")

# Schreiben: Endung bestimmt das Zielformat
export(ergebnis_tabelle, "output/ergebnis.xlsx")
export(ergebnis_tabelle, "output/ergebnis.csv")

# Format konvertieren in einer Zeile
convert("data/lieferung_2024q4.csv", "output/daten.xlsx")
```

`rio` eignet sich gut für Standardfälle. Für gezielte Kontrolle (Titelzeilen, Formatierung) bleiben `readxl` und `openxlsx` die genauere Wahl.

# PDF-Tabellen einlesen mit pdftools

Berichte kommen oft nur als PDF. `pdftools::pdf_text()` liefert je Seite eine lange Zeichenkette mit Zeilenumbrüchen. Daraus wird mit `stringr` eine Tabelle.

```
library(pdftools)

seiten <- pdf_text("data/lieferung_pdf.pdf")
length(seiten)      # ein Eintrag je Seite

zeilen <- strsplit(seiten[[1]], "\n")[[1]]
head(zeilen)
```

Jede Seite ist Text: Spalten sind durch mehrere Leerzeichen getrennt, Zeilen durch `\n`. Der Rest ist sauberes Parsen.

# PDF-Text mit stringr in eine Tabelle parsen

Skizze des Prinzips: Aus den Textzeilen werden Spalten, indem wir an Folgen von Leerzeichen trennen und die Werte in einen Tibble überführen. Die konkreten Muster passen Sie an Ihr PDF an.

```
library(stringr)
library(tibble)
library(readr)

# Nur Datenzeilen behalten (beginnen mit einem Bereichscode)
daten_zeilen <- zeilen %>%
  str_subset("^\\s*[A-Z]{1,2}\\s")

# An zwei oder mehr Leerzeichen in Felder zerlegen
felder <- str_split(str_trim(daten_zeilen), "\\s{2,}", simplify = TRUE)

tabelle <- tibble(
  bereich_code = felder[, 1],
  bereich_name = felder[, 2],
  wert         = parse_number(felder[, 3], locale = locale(decimal_mark = ","))
)
```

## Stolperfalle

PDF-Parsing ist fragil: Schon eine geänderte Spaltenbreite oder eine zusätzliche Fußzeile verschiebt das Raster. Nach dem Einlesen Zeilenzahl und Summen gegen das PDF prüfen und das Skript dokumentieren, damit es beim nächsten Durchlauf schnell nachjustiert werden kann.

# Datenbankzugriff mit DBI

Fachdatenbanken spricht R über `DBI` an: eine einheitliche Schnittstelle, der konkrete Treiber wird ausgetauscht. Für die Demo nutzen wir `RSQLite` gegen `edo_demo.sqlite` als Stellvertreter für EDO.

```
library(DBI)

con <- dbConnect(RSQLite::SQLite(), "data/edo_demo.sqlite")

dbListTables(con)
#> [1] "lieferung_2024q4" "lieferung_2025q1"

dbDisconnect(con)
```

`dbConnect()` öffnet die Verbindung, `dbListTables()` zeigt die Tabellen, `dbDisconnect()` schließt sauber. Dieselben Befehle gelten für SQLite, SQL Server oder Oracle, nur der Treiber wechselt.

# Abfragen: dbGetQuery und dplyr als DB-Backend

Sie können direkt SQL schicken oder die Datenbank mit `dplyr`-Verben ansprechen. `dplyr::tbl()` übersetzt die Pipe in SQL und holt erst bei `collect()` die Daten nach R.

```
library(DBI)
library(dplyr)

con <- dbConnect(RSQLite::SQLite(), "data/edo_demo.sqlite")

# Variante A: SQL direkt
dbGetQuery(con, "SELECT * FROM lieferung_2024q4 WHERE jahr = 2024")

# Variante B: dplyr gegen die DB, Berechnung läuft in der DB
tbl(con, "lieferung_2024q4") %>%
  filter(jahr == 2024, quartal == 4) %>%
  group_by(bereich_code) %>%
  summarise(summe = sum(wert, na.rm = TRUE)) %>%
  collect()

dbDisconnect(con)
```

## Praxis

Mit dem `dplyr`-Backend nutzen Sie dieselben Verben wie bei lokalen Tabellen. Die Aggregation läuft in der Datenbank, nur das fertige Ergebnis wandert nach R: das schont Speicher bei großen Beständen.

# EDO im Kurs: Demo statt Echtzugang

## Praxis

Für den Kurs brauchen Sie keinen echten EDO-Zugang. Wir nutzen `edo_demo.sqlite`, damit der Datenbank-Workflow vollständig offline und ohne Kundendaten funktioniert.

```
library(DBI)
con <- dbConnect(RSQLite::SQLite(), "data/edo_demo.sqlite")
dbListTables(con)
```

Die wichtige Idee ist **DBI** : Code für Tabellenlisten, Abfragen und **dplyr** -Pipelines bleibt gleichartig, auch wenn eine Organisation später selbst einen anderen Datenbanktreiber bereitstellt.

Öffentliche Statistikdatenbanken erreichen Sie analog: **eurostat** und **rsdmx** holen Eurostat- bzw. SDMX-Daten direkt nach R (nur als Hinweis; im Kurs arbeiten wir ohne Netzzugriff).

# Schnittstelle R und Excel: wann was

Excel bleibt im Haus, R übernimmt die wiederkehrende Verarbeitung. Die Kunst ist der saubere Übergang.

| Aufgabe                                 | Werkzeug           |
|-----------------------------------------|--------------------|
| Daten ansehen, einzelne Korrektur       | Excel              |
| Einlesen, Filtern, Rechnen, Joinen      | R                  |
| Wiederkehrende Eingabedatei verarbeiten | R                  |
| Ergebnis verteilen, weiterreichen       | Excel-Export aus R |

- R liest die Excel-Datei ein, statt Werte zu kopieren
- Das Skript ist die Dokumentation des Rechenwegs
- Das Ergebnis geht als formatierte `.xlsx` zurück

# Reproduzierbarer Einleseweg

## Stolperfalle

Copy-Paste zwischen Arbeitsmappen ist die häufigste Fehlerquelle: kein Protokoll, keine Wiederholbarkeit, stille Zahlendreher. Ein R-Skript dagegen ist jederzeit gleich ausführbar und prüfbar.

Derselbe Ablauf bei jedem Durchlauf, in drei Schritten:

```
library(readr)

roh      <- read_excel("data/lieferung_excel.xlsx",
                      sheet = "Daten", skip = 3)
ergebnis <- verarbeite(roh)           # dokumentierte Logik in einer Funktion
write_csv(ergebnis, "output/ergebnis.csv")
```

Lesen, rechnen, schreiben: ein Skript, das jeder nachvollziehen und erneut ausführen kann.

## Praxis

`readr::write_csv()` schreibt ohne Zeilennamen und mit UTF-8: keine `row.names` -Spalte, keine kaputten Umlaute. Das ist der Grund, base- `write.csv()` hier nicht zu verwenden.

# Teil IV: Fehlersuche und Ausgabe von Ergebnissen

# Worum es in diesem Teil geht

Zwei Fertigkeiten, die jeden R-Workflow tragfähig machen:

- **Teil A: Fehlersuche und -behebung.** Fehlermeldungen lesen, typische Anfängerfehler erkennen, mit `traceback()`, `browser()` und dem RStudio-Debugger gezielt suchen, defensiv programmieren.
- **Teil B: Ausgabe von Zwischen- und Teilergebnissen.** Ergebnisse interpretierbar zeigen ( `glimpse()`, `summary()`, schöne Tabellen mit `gt` ), als Grafik mit `ggplot2`, und als Datei speichern.

## Praxis

In der amtlichen Statistik zählt Nachvollziehbarkeit: Ein Rechenprozess ist erst dann fertig, wenn auch seine Zwischenergebnisse prüfbar und seine Ausgaben dokumentiert sind.

# Fehlermeldungen lesen und verstehen

R-Fehlermeldungen sind unschön, aber informativ. Lesen Sie sie von vorne nach hinten.

```
mean(umsatz)
#> Error in mean(umsatz) : object 'umsatz' not found
```

- `Error in mean(umsatz)` : **wo** der Fehler auftrat (in welchem Aufruf).
- `object 'umsatz' not found` : **was** schiefging (das Objekt existiert nicht).

Übersetzen Sie die Meldung in einen Satz: "In `mean()` wurde `umsatz` verlangt, aber es ist nicht definiert." Damit ist die Suche meist schon beantwortet.

# Fehler und Warnung sind nicht dasselbe

```
ergebnis <- as.numeric(c("12,5", "20,0"))  
#> Warning: NAs introduced by coercion  
ergebnis  
#> [1] NA NA
```

- **Error** : Die Ausführung **stoppt**. Es gibt kein Ergebnis.
- **Warning** : Die Ausführung **läuft weiter**, aber etwas ist verdächtig. Das Ergebnis kann falsch sein.

## Stolperfalle

Warnungen sind gefährlicher als Fehler, weil das Skript scheinbar durchläuft. Im Beispiel sind aus dem Dezimalkomma stillschweigend **NA** -Werte geworden. Prüfen Sie Warnungen immer, ignorieren Sie sie nie.

# Häufige Anfängerfehler (1): Tippfehler und Pfade

## Stolperfalle

**Objekt nicht gefunden / Funktion nicht gefunden.** Meist ein Tippfehler oder eine falsche Groß-/Kleinschreibung. R ist case-sensitiv: `Wert` und `wert` sind verschieden.

```
filterr(daten, jahr == 2024) # Error: could not find function "filterr"  
mean(Wert)                  # Error: object 'Wert' not found (Spalte heißt wert)
```

## Stolperfalle

**Falscher Pfad / falsches Arbeitsverzeichnis.** "cannot open file" heißt fast immer: relativer Pfad zeigt ins Leere.

```
read_csv("lieferung_2024q4.csv") # Error: ... does not exist  
read_csv("data/lieferung_2024q4.csv") # korrekt ab RStudio-Projektordner  
getwd() # zeigt das aktuelle Arbeitsverzeichnis
```

# Häufige Anfängerfehler (2): Daten und Pakete

## Stolperfalle

**Dezimalkomma.** Deutsche CSV nutzen oft `,` als Dezimaltrennzeichen. `read_csv()` erwartet `.` und liest die Spalte als Text. Lösung: `read_csv2()` oder `locale = locale(decimal_mark = ",")`.

## Stolperfalle

**factor statt character.** Eingelesene Texte können Faktoren sein. `paste0("Bereich ", bereich)` oder Vergleiche verhalten sich dann unerwartet. Mit `as.character()` umwandeln; `glimpse()` zeigt den Typ (`<fct>` vs. `<chr>`).

## Stolperfalle

**Paket nicht geladen.** "could not find function 'filter'" trotz korrekter Schreibweise: `library(dplyr)` bzw. `library(tidyverse)` fehlt am Skriptanfang. Im Zweifel `dplyr::filter()` voll qualifizieren.

# traceback(): wo ist der Fehler entstanden?

Nach einem Fehler zeigt `traceback()` die Aufrufkette (welche Funktion welche aufgerufen hat).

```
bws_anteil <- function(df) summe_pruefen(df$wert)
summe_pruefen <- function(x) sum(x) / laenge(x)    # Tippfehler: laenge

bws_anteil(daten)
#> Error in summe_pruefen(df$wert) : could not find function "laenge"

traceback()
#> 2: summe_pruefen(df$wert) at #1
#> 1: bws_anteil(daten)
```

Lesen Sie `traceback()` von unten nach oben: `bws_anteil()` rief `summe_pruefen()`, dort steckt der Fehler.

# browser(): Schritt für Schritt anhalten

`browser()` hält die Ausführung an dieser Stelle an. Sie landen in einer interaktiven Sitzung und sehen alle lokalen Variablen.

```
bws_anteil <- function(df) {  
  browser()           # Stopp hier  
  total <- sum(df$wert)  
  df$wert / total  
}
```

Im Browser-Modus: `n` (nächste Zeile), `c` (weiterlaufen), `Q` (abbrechen), oder einfach Variablennamen eintippen, um Werte zu prüfen.

# debug(): den Browser automatisch setzen

Statt `browser()` von Hand in den Code zu schreiben, markieren Sie eine ganze Funktion zum Debuggen.

```
debug(bws_anteil)      # ab jetzt: Stopp am Funktionsanfang  
bws_anteil(daten)      # landet sofort im Browser-Modus  
undebbug(bws_anteil)   # Markierung wieder aufheben
```

- `debug(f)` : setzt bei jedem Aufruf von `f()` einen `browser()` an den Funktionsanfang.
- `undebbug(f)` : hebt das wieder auf, ohne den Code zu ändern.

Praktisch, wenn die Funktion in einem fremden Paket steckt oder Sie den Quelltext nicht anfassen möchten.

# RStudio-Debugger und Breakpoints

Statt `browser()` in den Code zu schreiben, setzen Sie in RStudio einen **Breakpoint**:

- Klick links neben die Zeilennummer im Editor (roter Punkt erscheint), dann Skript mit `Source` ausführen.
- RStudio hält an, der **Environment**-Pane zeigt alle aktuellen Werte, der **Traceback**-Pane die Aufrufkette.
- Steuerung über die Buttons: Next, Step Into, Continue, Stop.

## Praxis

Für die Suche in einem mehrstufigen Rechenprozess ist der Breakpoint ideal: Sie halten genau vor dem verdächtigen Rechenschritt an und sehen die Zwischenwerte, ohne den Code zu ändern.

# Defensiv programmieren: stopifnot()

Prüfen Sie Annahmen früh und brechen Sie mit klarer Meldung ab, bevor falsche Ergebnisse entstehen.

```
bws_revision <- function(df) {  
  stopifnot(  
    "Spalte 'wert' fehlt"      = "wert" %in% names(df),  
    "wert muss numerisch sein" = is.numeric(df$wert),  
    "keine fehlenden Werte"   = !anyNA(df$wert)  
  )  
  # ... eigentliche Rechnung  
}
```

Schlägt eine Bedingung fehl, stoppt R mit der hinterlegten Meldung. Das ist besser als ein kryptischer Folgefehler drei Schritte später.

# message(), warning(), stop()

Drei Stufen, um den Nutzer zu informieren, in steigendem Ernst:

```
if (anyNA(df$wert)) {  
  message("Hinweis: ", sum(is.na(df$wert)), " fehlende Werte gefunden.")  
}  
if (any(df$wert < 0)) {  
  warning("Negative Bruttowertschöpfung in ", sum(df$wert < 0), " Zeilen.")  
}  
if (nrow(df) == 0) {  
  stop("Leerer Datensatz: Einlesen prüfen.")  
}
```

- `message()` : neutraler Hinweis (läuft weiter).
- `warning()` : Verdacht (läuft weiter, sammelt Warnungen).
- `stop()` : Abbruch (kontrolliert, mit eigener Meldung).

# Sinnvolle Prüfungen einbauen

Plausibilitätsprüfungen fangen Datenprobleme ab, die keine R-Fehler auslösen, aber das Ergebnis verfälschen.

```
library(dplyr)
pruef <- daten %>%
  summarise(
    n_zeilen    = n(),
    n_bereiche  = n_distinct(bereich_code),
    wert_min    = min(wert),
    wert_na     = sum(is.na(wert))
  )
pruef
```

## Praxis

Erwarten Sie 10 Wirtschaftsbereiche und keine negativen Werte? Dann prüft genau das Ihren Datenstand, bevor Sie weiterrechnen. Solche Prüfungen ersetzen den Excel-Blick "stimmen die Summen ungefähr?".

## Teil B: Zwischenergebnisse interpretierbar zeigen

Bevor Sie eine Tabelle gestalten, verschaffen Sie sich schnell einen Überblick.

```
glimpse(daten)
#> Rows: 280
#> Columns: 5
#> $ jahr          <int> 2018, 2018, 2018, ...
#> $ quartal       <int> 1, 2, 3, 4, ...
#> $ bereich_code  <chr> "A", "A", "A", ...
#> $ wert          <dbl> 5123.4, 5210.1, ...

summary(daten$wert)  # Min, Median, Mittelwert, Max, NA-Anzahl
```

- `glimpse()` : kompakter Blick auf Struktur und **Spaltentypen** (das `<int>` / `<chr>` / `<dbl>` ist Gold wert).
- `summary()` : Verteilungskennzahlen einer Spalte oder des ganzen Data Frames.

# Schöne Tabellen mit gt

Für Berichte und Ausdrücke formatiert `gt` eine Tabelle ansprechend.

```
library(gt); library(dplyr)
tab <- daten %>%
  filter(jahr == 2024, quartal == 4) %>%
  select(bereich_name, wert) %>%
  arrange(desc(wert)) %>%
  gt() %>%
  fmt_number(columns = wert, decimals = 1, sep_mark = ".", dec_mark = ",") %>%
  cols_label(bereich_name = "Wirtschaftsbereich", wert = "BWS (Mio. EUR)") %>%
  tab_header(title = "Bruttowertschöpfung 2024 Q4")
```

## Stolperfalle

Ist `gt` nicht verfügbar, nutzen Sie den Fallback `knitr::kable(df, digits = 1)`: schlichter, aber überall vorhanden und ideal für Konsole/Quarto.

# Grafik mit ggplot2: das Grundgerüst

Jedes ggplot besteht aus drei Bausteinen: Daten, **Aesthetics** ( `aes()` : welche Spalte auf welche Achse) und mindestens eine **Geometrie** ( `geom_*` ), mit `+` verbunden.

```
library(ggplot2)
ggplot(daten, aes(x = quartal, y = wert)) +
  geom_point()
```

- `ggplot(data, aes(...))` : Datensatz und Achsenzuordnung.
- `+ geom_*` : die sichtbare Darstellung (Punkte, Linien, Balken).
- Weitere `+`-Schichten: `labs()` , `facet_wrap()` , `theme_*` .

# Zeitreihe als Linie: BWS eines Bereichs

```
library(dplyr); library(ggplot2)
ein_bereich <- daten %>%
  filter(bereich_code == "F") %>%
  mutate(zeit = jahr + (quartal - 1) / 4)

ggplot(ein_bereich, aes(x = zeit, y = wert)) +
  geom_line(linewidth = 1, colour = "#1f5fa8") +
  labs(title = "BWS Baugewerbe (Bereich F)",
       x = "Jahr", y = "Mio. EUR") +
  theme_minimal()
```

## Praxis

Die Zeitreihe eines einzelnen Bereichs als Linie ist der schnellste Plausibilitätscheck: Brüche, Ausreißer oder ein "Knick" nach einer Revision fallen sofort auf.

# Balken: Bereiche vergleichen

`geom_col()` zeichnet Balken aus vorhandenen Werten. `reorder()` sortiert nach Höhe, `coord_flip()` dreht die Balken in die waagerechte Leserichtung.

```
library(dplyr); library(ggplot2)
q4 <- filter(daten, jahr == 2024, quartal == 4)

ggplot(q4, aes(x = reorder(bereich_code, wert), y = wert)) +
  geom_col(fill = "#1f5fa8") +
  coord_flip() +
  labs(x = "Bereich", y = "Mio. EUR")
```

# Kacheln mit facet\_wrap

`facet_wrap()` zerlegt eine Grafik in ein Kachelraster pro Gruppe: hier eine Zeitreihe je Wirtschaftsbereich.

```
library(ggplot2)
ggplot(daten, aes(jahr + (quartal - 1) / 4, wert)) +
  geom_line() +
  facet_wrap(~ bereich_code, scales = "free_y")
```

- `~ bereich_code`: eine Kachel je Ausprägung der Spalte.
- `scales = "free_y"`: eigene y-Achse je Kachel, damit kleine Bereiche nicht flachgedrückt werden.

# Ergebnisse als Datei speichern: Grafiken

`ggsave()` schreibt den zuletzt erzeugten (oder einen benannten) Plot in eine Datei. Format ergibt sich aus der Endung.

```
p <- ggplot(ein_bereich, aes(zeit, wert)) + geom_line()

ggsave("output/bws_F.png", plot = p, width = 8, height = 5, dpi = 150)
ggsave("output/bws_F.pdf", plot = p, width = 8, height = 5)
```

- `.png` : für Folien, E-Mail, Web.
- `.pdf` : vektorbasiert, für Druck und Berichte.
- `width / height` in Zoll, `dpi` für die Pixelauflösung von PNG.

# Ergebnisse als Datei speichern: Tabellen

```
library(readr)
write_csv(q4, "output/bws_2024q4.csv") # einfacher Austausch, UTF-8

library(openxlsx)
write.xlsx(
  list("BWS_2024Q4" = q4, "Prüfsummen" = pruef), # je Listenelement ein Blatt
  file = "output/ergebnisse.xlsx"
)
```

- `readr::write_csv()` : schlankes, offenes Format. Für deutsche Tools ggf. `write_csv2()` (Semikolon, Komma).
- `openxlsx::write.xlsx()` : echtes Excel, mehrere Blätter, ohne Java. Für Übergaben an Excel-Kolleginnen.

## Praxis

Eine `gt` -Tabelle speichern Sie mit `gtsave(tab, "output/tab.html")` : das ist der robuste Standard, der ohne Zusatzwerkzeuge funktioniert. Der Export als Bild ( `gtsave(tab, "output/tab.png")` ) braucht zusätzlich das Paket `webshot2` bzw. `chromote` und ein installiertes Headless-Chrome. Im Zweifel HTML wählen.

# Zwischenergebnisse prüfbar machen

## Praxis

Bauen Sie in jeden mehrstufigen Rechenprozess feste Prüfpunkte ein:

- **Kontrolltabellen** nach jedem Schritt: `summarise()` mit Zeilenzahl, Bereichszahl, Summen je Jahr. Mit `write_csv()` als Protokoll ablegen.
- **Plausibilitätsplots**: Zeitreihe je Bereich vor und nach einer Revision übereinanderlegen, um Sprünge zu erkennen.
- Prüfsummen mit dem vorigen Datenstand vergleichen, bevor das Ergebnis freigegeben wird.

So bleibt der Prozess auch für die Vertretung und die Revision nachvollziehbar: Jeder Schritt hat ein sichtbares, ablegbares Ergebnis.

# Zusammenfassung Teil IV

- Fehlermeldungen sagen **wo** und **was**; `Warning` ist gefährlicher als `Error`, weil der Code weiterläuft.
- Typische Stolperfallen: Tippfehler, falsches Arbeitsverzeichnis, Dezimalkomma, factor statt character, fehlendes `library()`.
- Gezielt suchen: `traceback()`, `browser()` / `debug()`, RStudio-Breakpoints.
- Defensiv programmieren: `stopifnot()`, `message()` / `warning()` / `stop()`, Plausibilitätsprüfungen.
- Ergebnisse zeigen: `glimpse()` / `summary()`, `gt` (Fallback `knitr::kable`), `ggplot2`.
- Ergebnisse speichern: `ggsave()`, `readr::write_csv()`, `openxlsx::write.xlsx()`.

# Teil V: Wartbare R-Projekte, manuelle Eingriffe und If-Logik

# Von der Bastellösung zum Produktionsprozess

Bis hierhin haben Sie Daten eingelesen, aufbereitet und ausgegeben. Für einen wiederkehrenden Produktionsprozess reicht ein einzelnes langes Skript aber nicht.

- Reproduzierbar: gleicher Code, gleiche Daten, gleiches Ergebnis, auch nächstes Quartal
- Modular: einzelne Teilprozesse austauschbar, ohne alles neu zu schreiben
- Nachvollziehbar: jeder manuelle Eingriff ist dokumentiert, nicht still in einer Zelle versteckt

## Praxis

Ein VGR-Rechenprozess läuft jedes Quartal erneut, oft jahrelang und in wechselnder Besetzung. Was heute funktioniert, muss in zwei Jahren von einer Vertretung verstanden und korrekt wiederholt werden können.

# Projektskelett für die Produktion

Vertiefung zu Teil I: ein festes Gerüst statt loser Skripte.

```
projekt/  
  projekt.Rproj      # RStudio-Projekt: definiert das Arbeitsverzeichnis  
  data/              # Eingangsdaten, nie verändern  
  R/                 # Skripte und Funktionen  
  output/            # erzeugte Tabellen, Grafiken, Dateien  
  renv.lock          # eingefrorene Paketversionen
```

- Ein RStudio-Projekt pro Prozess: Doppelklick öffnet Pfade und Verlauf konsistent
- Trennung Eingang / Code / Ausgabe macht klar, was Quelle und was Ergebnis ist

# Pfade robust halten mit `here`

`setwd("C:/Users/...")` bricht, sobald jemand anderes das Projekt öffnet. `here` löst Pfade immer relativ zur Projektwurzel auf.

```
library(here)

# here() findet die Projektwurzel (am .Rproj erkannt)
daten <- readr::read_csv(here("data", "lieferung_2024q4.csv"))

readr::write_csv(ergebnis, here("output", "bws_2024q4.csv"))
```

## Stolperfalle

Absolute Pfade und `setwd()` sind die häufigste Ursache dafür, dass ein Skript "bei mir lief, bei dir nicht".  
Verwenden Sie konsequent `here()` oder Pfade relativ zu `data/`.

# Reproduzierbarkeit mit `renv`

`renv` friert die genutzten Paketversionen pro Projekt ein, damit ein Update von `dplyr` Ihren Quartalsprozess nicht still verändert.

```
renv::init()      # einmal: eigenes Paket-Verzeichnis fürs Projekt
renv::snapshot()  # schreibt renv.lock (aktuelle Versionen)
renv::restore()   # stellt genau diese Versionen wieder her
```

- `renv.lock` kommt mit ins Projekt (und in die Versionsverwaltung)
- Vertretung oder Nachfolge bekommt mit `renv::restore()` exakt Ihre Umgebung

## Praxis

Für die amtliche Statistik zählt nicht nur das Ergebnis, sondern die Nachweisbarkeit, wie es zustande kam. Eingefrorene Versionen sind Teil dieser Nachweiskette.

# Eigene Funktionen schreiben

Sobald Sie denselben Schritt mehrfach brauchen, wird er eine Funktion: ein Name, Argumente, ein Rückgabewert.

```
# Wachstumsrate gegenüber Vorjahresquartal, in Prozent
wachstumsrate <- function(wert, wert_vorjahr) {
  (wert / wert_vorjahr - 1) * 100
}

wachstumsrate(105, 100)    # Rückgabewert: 5
```

- Argumente: die Eingaben ( `wert` , `wert_vorjahr` )
- Rückgabewert: der letzte ausgewertete Ausdruck (oder `return(...)` )

# DRY: Don't Repeat Yourself

Kopierte Codeblöcke driften auseinander: Sie korrigieren einen, vergessen den anderen. Eine Funktion hat genau eine Stelle zum Pflegen.

Vorher: dreimal fast gleich

```
a$rate <- (a$w / a$wv - 1) * 100  
b$rate <- (b$w / b$wv - 1) * 100  
c$rate <- (c$w / c$wv - 1) * 100
```

Nachher: eine Quelle der Wahrheit

```
a$rate <- wachstumsrate(a$w, a$wv)  
b$rate <- wachstumsrate(b$w, b$wv)  
c$rate <- wachstumsrate(c$w, c$wv)
```

## Stolperfalle

Wenn Sie beim Schreiben Code kopieren und nur Zahlen oder Namen ändern, ist das fast immer das Signal für eine Funktion.

# Skript in Funktionen und Module zerlegen

Ein Produktionsprozess besteht aus klar benannten Teilschritten. Legen Sie diese in eigene Dateien und laden Sie sie mit `source()`.

```
# R/funktionen_einlesen.R, R/funktionen_pruefen.R, ...
source(here("R", "funktionen_einlesen.R"))
source(here("R", "funktionen_pruefen.R"))

roh      <- daten_einlesen(here("data", "lieferung_2024q4.csv"))
geprueft <- plaesibilitaet_pruefen(roh)
```

- Das Hauptskript liest sich dann wie ein Inhaltsverzeichnis des Prozesses
- Einzelne Teilprozesse lassen sich austauschen, ohne den Rest anzufassen

# Blockweiser Ersatz von Teilprozessen

Modularität zahlt sich aus, wenn sich genau ein Schritt ändert: neue Klassifikation, andere Quelle, korrigierte Regel.

- Funktion `daten_einlesen()` ersetzen, weil das Format wechselt
- Funktion `plausibilitaet_pruefen()` erweitern, weil eine Regel dazukommt
- Der übrige Prozess bleibt unverändert und nachweislich gleich

## Praxis

Typischer VGR-Fall: Die Reklassifikation ändert sich, der Rest der Rechenkette nicht. Mit getrennten Funktionen tauschen Sie nur den betroffenen Block, statt das ganze Skript zu riskieren.

# Pipeline-Idee mit `targets`

Bei vielen Teilschritten lohnt eine Pipeline: `targets` merkt sich, welcher Schritt von welchem abhängt, und rechnet nur das Geänderte neu.

```
# _targets.R (Skizze)
library(targets)
list(
  tar_target(roh,      daten_einlesen("data/lieferung_2024q4.csv")),
  tar_target(geprueft, plausibilitaet_pruefen(roh)),
  tar_target(ergebnis, aggregieren(geprueft))
)
# tar_make() rechnet nur veraltete Ziele neu
```

- Ändert sich nur `aggregieren()`, bleibt `roh` und `geprueft` unangetastet
- In dieser Schulung als Ausblick: das Prinzip zählt, nicht die volle Einrichtung

# Verzweigungen mit `if` und `else`

`if` / `else` steuert den Ablauf anhand einer einzelnen Bedingung (Länge 1, TRUE oder FALSE).

```
pruefe_daten <- function(df) {  
  if (nrow(df) == 0) {  
    stop("Leerer Datensatz: Datei prüfen.")  
  } else if (anyNA(df$wert)) {  
    warning("Fehlende Werte in der Spalte wert.")  
  }  
  df  
}
```

## Stolperfalle

`if` erwartet genau einen Wahrheitswert. Eine ganze Spalte ( `if (df$wert > 0)` ) führt zu Fehler oder Warnung. Für spaltenweise Entscheidungen nutzen Sie `ifelse()` oder `case_when()` .

# Vektorisiert entscheiden mit `ifelse()`

`ifelse()` wertet eine Bedingung für jede Zeile aus und liefert einen Vektor zurück: ideal für eine neue Spalte.

```
library(dplyr)

geprueft <- daten %>%
  mutate(
    plausibel = ifelse(wert >= 0, "ok", "negativ"),
    wert      = ifelse(wert < 0, NA_real_, wert)    # negative Werte verwerfen
  )
```

- Eine Bedingung, zwei Ausgänge (wahr / falsch)
- Achten Sie auf einheitliche Typen in beiden Zweigen (z. B. `NA_real_`)

## Mehr als zwei Fälle: `dplyr::case_when()`

Für mehrere, geordnete Validierungsregeln ist `case_when()` lesbarer als verschachtelte `ifelse()`.

```
geprueft <- daten %>%  
  mutate(  
    befund = case_when(  
      is.na(wert) ~ "fehlend",  
      wert < 0 ~ "negativ",  
      wert > 5 * median(wert, na.rm = TRUE) ~ "auffällig_hoch",  
      TRUE ~ "ok"  
    )  
  )
```

- Regeln werden von oben nach unten geprüft, die erste passende gewinnt
- `TRUE ~ ...` ist der Auffangfall für alles Übrige

# Manuelle Eingriffe: das Problem

In Excel wird ein Wert oft einfach in der Zelle überschrieben. Nach drei Monaten weiß niemand mehr, welcher Wert geändert wurde, von wem und warum.

## Stolperfalle

Stille Handkorrektur direkt im Datenobjekt ( `df$wert[42] <- 22` ) ist in einem Produktionsprozess die gefährlichste Stelle: nicht reproduzierbar, nicht prüfbar, beim nächsten Lauf überschrieben.

- Der Eingriff verschwindet beim nächsten Neueinlesen der Quelldatei
- Keine Begründung, kein Urheber, kein Datum: nicht revisionssicher

# Manuelle Eingriffe als dokumentierte Override-Tabelle

Trennen Sie die Korrektur von den Daten: eine eigene CSV mit Schlüssel, neuem Wert und den Spalten wer / wann / warum.

```
# data/overrides.csv  
# jahr,quartal,bereich_code,neuer_wert,bearbeiter,datum,grund  
# 2024,4,F,49200,m.mueller,2026-01-15,"Nachmeldung Baugewerbe Q4"  
overrides <- readr::read_csv(here("data", "overrides.csv"))
```

- Die Quelldatei bleibt unverändert, der Eingriff steht versioniert daneben
- Jede Korrektur trägt Grund, Bearbeiter und Datum

# Overrides absichern

Vor dem Anwenden prüfen Sie, ob ein Schlüssel doppelt vorkommt. Sonst würde ein `left_join` Zeilen vervielfachen und das Ergebnis still verfälschen.

```
library(dplyr)

dup <- overrides %>%
  count(bereich_code, jahr, quartal) %>%
  filter(n > 1)

if (nrow(dup) > 0) {
  stop("Doppelte Override-Schlüssel: ", nrow(dup), " Fälle prüfen.")
}
```

- Ein Schlüssel ( `bereich_code` , `jahr` , `quartal` ) darf höchstens einen manuellen Wert tragen
- Der Abbruch zwingt zur Klärung, bevor falsche Zahlen weiterlaufen

# Overrides anwenden per Join

Die Korrektur wird beim Lauf über einen `left_join` eingespielt, nicht von Hand eingetragen.

```
ergebnis <- daten %>%  
  left_join(overrides, by = c("bereich_code", "jahr", "quartal")) %>%  
  mutate(  
    final_wert      = ifelse(is.na(neuer_wert), wert, neuer_wert),  
    override_aktiv = !is.na(neuer_wert)  
  )
```

- `final_wert` nimmt den manuellen Wert nur dort, wo einer hinterlegt ist
- `override_aktiv` markiert jede gegriffene Korrektur für die Prüfung

# VGR-Plausibilitätsregeln als Status

## Praxis

Korrektur- und Plausibilitätsregeln der VGR lassen sich als `case_when` fassen und mit der Override-Logik verbinden: so erhält jeder Wert einen Status, der zeigt, wie er zustande kam.

```
geprueft <- ergebnis %>%  
  mutate(  
    status = case_when(  
      override_aktiv      ~ "manuell_korrigiert",  
      is.na(final_wert)   ~ "fehlend",  
      final_wert < 0      ~ "regelverstoß_negativ",  
      final_wert == 0     ~ "prüfen_null",  
      TRUE                ~ "automatisch_ok"  
    )  
  )
```

# Audit-Tabelle ausgeben

Aus den klassifizierten Werten wird ein Protokoll: nur die prüfrelevanten Spalten, als CSV gespeichert.

```
audit <- geprueft %>%  
  select(bereich_code, jahr, quartal, wert, neuer_wert,  
         final_wert, status, grund, bearbeiter, datum)  
  
readr::write_csv(audit, here("output", "audit_2024q4.csv"))
```

- Die Audit-Tabelle dokumentiert je Zeile, ob automatisch oder von Hand entschieden wurde
- Sie wandert nach `output/` und bleibt als Nachweis erhalten

# **Teil VI: Zeitreihen, Schleifen, Gliederungsänderungen und mehrere Datenstände**

# Worum es in diesem Teil geht

## Praxis

Der Arbeitsalltag besteht aus wiederkehrenden Rechenschritten auf Zeitreihen: ein neues Quartal kommt hinzu, ausgewählte Jahre werden revidiert, mehrere Datenstände müssen über Schlüssel zusammengeführt und gegen Aggregate geprüft werden, und gelegentlich ändert sich die Gliederung.

- Zeit in R sauber darstellen: Jahr und Quartal als geordnete Zeitachse
- Über Jahre und Bereiche iterieren: `for` verstehen, dann `purrr::map()`
- Zeitreihen ableiten: Wachstumsraten, Index, gleitende Summe
- Ausgewählte Jahre aktualisieren, ohne den Rest anzufassen
- Mehrere Datenstände per Join verknüpfen und gegen Aggregate abstimmen
- Gliederungsänderungen über eine Korrespondenztabelle abbilden

# Zeit in R: jahr und quartal

In den Daten ist Zeit auf zwei Spalten verteilt: `jahr` (int) und `quartal` (int 1 bis 4). Für viele Auswertungen reicht das, sortiert wird nach beiden Spalten.

```
library(here)
bws <- readr::read_csv(here("data", "lieferung_2024q4.csv"))

bws %>%
  filter(bereich_code == "BE") %>%
  arrange(jahr, quartal) %>%
  select(jahr, quartal, wert)
```

## Stolperfalle

Sortieren Sie Zeitreihen immer explizit ( `arrange(jahr, quartal)` ). Verlassen Sie sich nie auf die Reihenfolge in der Datei: ein Join oder ein `group_by()` kann sie verändern, und `lag()` rechnet dann falsch.

# Quartale als echte Zeitachse: lubridate und tsibble

Für Zeitreihen-Logik (gleiche Abstände, Lücken erkennen) hilft ein echter Zeitindex.

`tsibble::yearquarter()` macht aus Jahr und Quartal einen geordneten Quartalswert.

```
library(tsibble); library(lubridate)

bws_zeit <- bws %>%
  mutate(
    datum = make_date(jahr, quartal * 3 - 2, 1), # lubridate: Quartalsbeginn
    yq     = yearquarter(datum)                 # tsibble: 2024 Q4 usw.
  )
```

- `make_date()` (lubridate) baut aus Jahr/Monat/Tag ein Datum
- `yearquarter()` (tsibble) ordnet Quartale korrekt, auch über Jahresgrenzen
- Eine `tsibble` mit `index = yq` und `key = bereich_code` ist die tidy Form für Zeitreihen

# Schleifen verstehen: for

Eine `for`-Schleife wiederholt einen Block für jeden Wert. Nützlich zum Verstehen, oft aber unhandlich, weil man das Ergebnis selbst einsammeln muss.

```
jahre <- 2018:2024
summen <- numeric(length(jahre))          # Ergebnis vorbereiten

for (i in seq_along(jahre)) {
  jahr_i <- jahre[i]
  summen[i] <- sum(bws$wert[bws$jahr == jahr_i])
}
summen
```

## Stolperfalle

Häufiger Anfängerfehler: das Ergebnis mit `c()` in jeder Runde wachsen lassen (`x <- c(x, neu)`). Bei vielen Durchläufen wird das langsam. Ergebnis vorab dimensionieren oder gleich `purrr::map()` nutzen.

# Idiomatischer: `purrr::map()` und `list_rbind()`

`purrr::map()` wendet eine Funktion auf jedes Element an und gibt eine Liste zurück.

`list_rbind()` bindet diese Liste anschließend zeilenweise zu einem Data Frame zusammen.

```
library(purrr)

# je Bereich die Jahressumme als ein Tibble, dann untereinander
jahressummen <- bws %>%
  split(bws$bereich_code) %>%
  map(\(df) tibble(
    bereich_code = df$bereich_code[1],
    summe = sum(df$wert)
  )) %>%
  list_rbind()
```

Lesart: `\(df) ...` ist eine kurze anonyme Funktion. `map()` ruft sie je Teil auf, `list_rbind()` stapelt die Ergebnisse.

## Stolperfalle

Das frühere `map_dfr()` machte beides in einem Schritt, ist aber superseded. Standard ist heute `map(...) %>% list_rbind()` : ein klarer Schritt zum Rechnen, einer zum Binden.

# map() über Jahre oder Bereiche

Typisches Muster: dieselbe Auswertung für mehrere Jahre laufen lassen.

```
# eine Funktion, die ein Jahr auswertet
auswertung_jahr <- function(j) {
  bws %>%
    filter(jahr == j) %>%
    summarise(jahr = j, summe = sum(wert))
}

# über alle Jahre iterieren und stapeln
2018:2024 %>%
  map(auswertung_jahr) %>%
  list_rbind()
```

# map() über mehrere Quelldateien

Derselbe Mechanismus skaliert von einer auf viele Eingabedateien.

```
# Pfade aller Quelldateien einsammeln
pfade <- list.files(here::here("data"),
                   pattern = "lieferung_.*csv",
                   full.names = TRUE)

# jede Datei lesen, dann zeilenweise stapeln
alle_staende <- pfade %>%
  map(readr::read_csv) %>%
  list_rbind()
```

## Praxis

`list.files()` liefert die Pfade, `map(read_csv)` liest jede Datei zu einem Tibble, `list_rbind()` stapelt sie. So wächst der Prozess ohne Codeänderung mit jeder neuen Quelldatei.

# Wachstumsraten und Index mit dplyr::lag

Für abgeleitete Zeitreihen brauchen Sie den Vorperiodenwert. `dplyr::lag()` holt ihn, immer innerhalb von `group_by()` und nach `arrange()`.

```
bws_raten <- bws %>%  
  arrange(bereich_code, jahr, quartal) %>%  
  group_by(bereich_code) %>%  
  mutate(  
    vorquartal = lag(wert),  
    wachstum   = (wert / vorquartal - 1) * 100,      # Veränderung zum Vorquartal  
    vorjahr    = lag(wert, 4),  
    vj_rate    = (wert / vorjahr - 1) * 100          # Vorjahresvergleich (4 Quartale)  
  ) %>%  
  ungroup()
```

## Stolperfalle

`lag()` ohne vorheriges `group_by(bereich_code)` zieht den Wert vom letzten Bereich heran. Das erste Quartal jeder Reihe hat zwangsläufig `NA`: das ist korrekt, kein Fehler.

# Index und gleitende Summe

Ein Index normiert eine Reihe auf ein Basisjahr (Basis = 100). Eine gleitende Summe glättet, etwa die Vier-Quartals-Summe als Jahreswert.

```
bws_index <- bws %>%  
  arrange(bereich_code, jahr, quartal) %>%  
  group_by(bereich_code) %>%  
  mutate(  
    basis = wert[jahr == 2020 & quartal == 1][1],    # Basiswert je Bereich  
    index = wert / basis * 100,  
    gleit_summe = wert + lag(wert) + lag(wert, 2) + lag(wert, 3) # 4 Quartale  
  ) %>%  
  ungroup()
```

## Praxis

Für echte gleitende Fenster ist `slider::slide_dbl()` komfortabler (`slide_dbl(wert, sum, .before = 3)`). Für den Einstieg reicht die Summe aus `lag()`-Werten und bleibt gut lesbar.

# Ein neues Quartal anhängen

Iterative Prozesse hängen den jeweils aktuellen Stand an. Hier am Beispiel einer fortgeschriebenen Annahme (in der Praxis kommen die Werte aus dem neuen Datenstand).

```
letztes <- bws %>% filter(jahr == 2024, quartal == 4)

neu_2025q1 <- letztes %>%
  mutate(jahr = 2025, quartal = 1,
         wert = round(wert * 1.008, 1)) # Platzhalter-Fortschreibung

bws_fortgeschrieben <- bind_rows(bws, neu_2025q1) %>%
  arrange(jahr, quartal, bereich_code)
```

# Ausgewählte Jahre aktualisieren: rows\_update()

Bei einer Revision sollen nur bestimmte Jahre (hier 2023 und 2024) durch revidierte Werte ersetzt werden, alles andere bleibt unverändert. `dplyr::rows_update()` ersetzt gezielt anhand der Schlüssel.

```
vintage2 <- readr::read_csv(here::here("data", "lieferung_2025q1.csv"))

# nur die revidierten Zeilen 2023/2024 aus dem neuen Datenstand
revidierte_zeilen <- vintage2 %>%
  filter(jahr %in% c(2023, 2024)) %>%
  select(jahr, quartal, bereich_code, wert)

bws_aktuell <- bws %>%
  rows_update(revidierte_zeilen,
             by = c("jahr", "quartal", "bereich_code"))
```

## Stolperfalle

`rows_update()` verlangt, dass jede zu ersetzende Zeile genau einmal über die Schlüssel getroffen wird. Doppelte Schlüssel oder ein fehlender Treffer brechen mit Fehler ab: gewollt, denn das deckt Schlüsselprobleme sofort auf.

# Aktualisieren per Join als Alternative

Ohne `rows_update()` geht dasselbe als `left_join` plus `coalesce` : revidierten Wert nehmen, wo vorhanden, sonst den alten behalten.

```
bws_aktuell2 <- bws %>%  
  left_join(revidierte_zeilen,  
            by = c("jahr", "quartal", "bereich_code"),  
            suffix = c("", "_rev")) %>%  
  mutate(wert = coalesce(wert_rev, wert)) %>%    # Revision schlägt Altwert  
  select(-wert_rev)
```

So sehen Sie zugleich, welche Zeilen überhaupt revidiert wurden ( `!is.na(wert_rev)` ).

# Mehrere Datenstände verknüpfen: die Join-Familie

Die Datenstände werden über die Schlüssel `jahr`, `quartal`, `bereich_code` verknüpft. Welcher Join, hängt von der Frage ab.

```
schluessel <- c("jahr", "quartal", "bereich_code")

# alle Altzeilen, neue Werte daneben (auch ohne Treffer)
left_join(bws, vintage2, by = schluessel, suffix = c("_alt", "_neu"))

# nur Zeilen, die in beiden Datenständen vorkommen
inner_join(bws, vintage2, by = schluessel)

# Zeilen aus vintage2, die in bws fehlen (z. B. das neue 2025Q1)
anti_join(vintage2, bws, by = schluessel)
```

- `left_join`: behält links alles, ergänzt rechts
- `inner_join`: nur gemeinsame Schlüssel
- `anti_join`: was rechts fehlt, ideal zum Aufspüren von Neuem oder Lücken

# Schlüsseltreue und Duplikate

## Stolperfalle

Vor jedem Join die Schlüssel prüfen. Ein doppelter Schlüssel auf einer Seite vervielfacht Zeilen (Kreuzprodukt), und Summen stimmen plötzlich nicht mehr.

```
bws %>% count(jahr, quartal, bereich_code) %>% filter(n > 1) # muss leer sein

# dplyr warnt aktiv vor unerwarteten Mehrfachtreffern:
left_join(bws, vintage2, by = schluessel,
          relationship = "one-to-one")
```

Stimmt die Spaltenschreibweise nicht überein ( `bereich_code` vs. `Bereich` ), findet der Join nichts. Schlüssel zuerst vereinheitlichen.

# Reconciliation: Teilbereiche gegen Aggregat prüfen

Reconciliation (Abstimmung): die Summe der Wirtschaftsbereiche muss dem Gesamttaggregat entsprechen. Das Aggregat stammt aus einer eigenen Quelle, nicht aus den Teilbereichen. Diese Kontrolle gehört in jeden Produktionslauf.

```
# Aggregat je Quartal aus den Teilbereichen bilden
agg <- bws %>%
  group_by(jahr, quartal) %>%
  summarise(summe_bereiche = sum(wert), .groups = "drop")

# separat geliefertes Gesamttaggregat: Spalten jahr, quartal, wert_gesamt
gesamt <- readr::read_csv(here::here("data", "gesamt_aggregat.csv"))

# Teilbereichssumme gegen das eigenständige Gesamttaggregat abgleichen
kontrolle <- agg %>%
  left_join(gesamt, by = c("jahr", "quartal")) %>%
  mutate(abweichung = summe_bereiche - wert_gesamt) %>%
  filter(abs(abweichung) > 0.5)      # Toleranz für Rundung

stopifnot(nrow(kontrolle) == 0)      # bricht ab, wenn etwas nicht stimmt
```

## Praxis

Das Gesamttaggregat kommt aus einer separaten Quelle. Daher stimmt es mit der aufsummierten Teilbereichsreihe nur bis auf Rundung überein: deshalb der Vergleich mit Toleranz ( `abs(abweichung) > 0.5` ) statt exakter Gleichheit. `stopifnot()` macht die Kontrolle verbindlich und stoppt den Lauf, statt falsche Zahlen weiterzureichen.

# Gliederungsänderung: die Korrespondenztabelle

Ändert sich die Gliederung, beschreibt eine Korrespondenztabelle den Übergang. In den Daten wird **GI** in **G**, **H**, **I** aufgeteilt; alle übrigen Bereiche bleiben 1:1.

```
korresp <- readr::read_csv(here::here("data", "klassifikation_korrespondenz.csv"))
korresp %>% filter(alt_code == "GI")
```

| # | alt_code | neu_code | neu_name            | anteil |
|---|----------|----------|---------------------|--------|
| # | GI       | G        | Handel              | 0.52   |
| # | GI       | H        | Verkehr und Lagerei | 0.36   |
| # | GI       | I        | Gastgewerbe         | 0.12   |

Spalten: **alt\_code**, **neu\_code**, **neu\_name**, **anteil**. Die **anteil**-Werte je **alt\_code** summieren sich zu 1.

# Vor der Reklassifikation: vollständige Korrespondenz prüfen

Ein `inner_join` mit der Korrespondenztabelle verliert still jede Zeile, deren `bereich_code` dort fehlt. Prüfen Sie das vorher explizit mit `anti_join()`.

```
# Codes in bws, die in der Korrespondenztabelle keinen Eintrag haben
fehlende <- bws %>%
  anti_join(korresp, by = c("bereich_code" = "alt_code"))

if (nrow(fehlende) > 0) {
  stop("Codes ohne Korrespondenz: ",
       paste(unique(fehlende$bereich_code), collapse = ", "))
}
```

## Stolperfalle

Stiller Datenverlust ist die häufigste Reklassifikationsfalle: ohne diese Prüfung fallen Bereiche unbemerkt aus der Summe heraus, und das Aggregat stimmt nicht mehr. Die Prüfung gehört vor jeden `inner_join` mit einer Schlüsseltabelle.

# Reklassifikation anwenden: alte in neue Gliederung überführen

Nach der Prüfung: Join über `alt_code = bereich_code`, den Wert mit `anteil` splitten und auf die neuen Codes aggregieren.

```
bws_neu <- bws %>%  
  inner_join(korresp, by = c("bereich_code" = "alt_code"),  
             relationship = "many-to-many") %>% # GI trifft auf G/H/I, je Quartal  
  mutate(wert_neu = wert * anteil) %>%          # GI auf G/H/I verteilen  
  group_by(jahr, quartal, neu_code, neu_name) %>%  
  summarise(wert = sum(wert_neu), .groups = "drop") # erst beim Export runden
```

## Stolperfalle

`relationship = "many-to-many"` benennt die Aufteilung bewusst: ein `alt_code` (GI) trifft auf mehrere `neu_code` und kommt in mehreren Quartalen vor. Ohne die Angabe warnt dplyr. Kontrolle danach: die Gesamtsumme je Quartal muss vorher und nachher übereinstimmen, denn die Anteile summieren sich zu 1. Vergleichen Sie mit Toleranz ( `abs(diff) > 0.5` ), nicht exakt: durch das Splitten entstehen kleine Rundungsdifferenzen. Runden Sie erst beim Export oder in der Darstellung.

# Alte und neue Klassifikation nebeneinander

Während der Umstellung werden oft beide Gliederungen parallel benötigt. Halten Sie die Zuordnung in einer Tabelle, nicht im Kopf, und behalten Sie eine Spalte mit der Herkunft.

```
bruecke <- bws %>%  
  inner_join(korresp, by = c("bereich_code" = "alt_code"),  
             relationship = "many-to-many") %>%  
  mutate(wert_neu = wert * anteil) %>%  
  select(jahr, quartal,  
         alt_code = bereich_code, neu_code, neu_name,  
         wert_alt = wert, anteil, wert_neu)
```

Auch dieser `inner_join` läuft über `alt_code` : prüfen Sie zuvor mit demselben `anti_join()` - Schritt auf fehlende Codes, sonst fehlen einzelne Bereiche still in der Brückentabelle. Sie erlaubt jederzeit den Rückweg und macht die feinere Aufteilung nachvollziehbar.

# Datenstände und Klassifikationswechsel im VGR-Alltag

## Praxis

Jeder Datenstand (englisch: Vintage) hat seinen eigenen Zeitstempel: 2024Q4, dann 2025Q1 mit revidierten Vorjahren plus neuem Quartal. Bewahren Sie die Datenstände getrennt auf, statt sie zu überschreiben: nur so sind Revisionen später nachvollziehbar.

Klassifikationswechsel (etwa NACE-Revisionen) laufen über Korrespondenztabelle mit Anteilen. Die Tabelle ist die dokumentierte Wahrheit des Übergangs und gehört versioniert ins Projekt.

# **Teil VII: Alternative Quellen vergleichen und Abweichungen analysieren**

# Worum es in diesem Teil geht

Im VGR-Bereich liegt selten nur ein Datenstand vor: ein älterer Datenstand, ein revidierter Datenstand, eine Excel-Version vom Fachbereich, ein Export aus der Datenbank.

- Denselben Prozess auf alternative Quelldateien anwenden (Vintage 1 gegen Vintage 2)
- Beide Stände per Join zusammenführen und Abweichungen quantifizieren
- Neue und weggefallene Schlüssel sauber sichtbar machen ( `anti_join()` )
- Objekte und Tabellen direkt vergleichen mit `waldo` und `diffdf`
- Einen kompakten, prüfbaren Abweichungs-Report exportieren

## Praxis

Ziel: nicht „Excel sagt etwas anderes“, sondern ein nachvollziehbarer, reproduzierbarer Vergleich, der zeigt, welche Zelle sich um wie viel geändert hat.

# Zwei Datenstände desselben Datensatzes

Im Datensatz liegen zwei Datenstände („Vintages“) derselben Zeitreihe vor:

- `lieferung_2024q4.csv` (Vintage 1): Bruttowertschöpfung 2018 bis 2024, 280 Zeilen
- `lieferung_2025q1.csv` (Vintage 2): 2023/2024 leicht revidiert, 2025Q1 neu, 290 Zeilen

Beide haben dieselben Spalten: `jahr`, `quartal`, `bereich_code`, `bereich_name`, `wert` (Mio. EUR, jeweilige Preise).

```
library(tidyverse)

v1 <- read_csv("data/lieferung_2024q4.csv")
v2 <- read_csv("data/lieferung_2025q1.csv")

dim(v1)  # 280    5
dim(v2)  # 290    5
```

# Den Einleseprozess parametrisieren

Statt den Importcode zu kopieren, kapseln Sie ihn in einer Funktion. Der Dateipfad wird zum Parameter.

```
daten_einlesen <- function(pfad) {  
  read_csv(  
    pfad,  
    col_types = cols(  
      jahr          = col_integer(),  
      quartal       = col_integer(),  
      bereich_code  = col_character(),  
      bereich_name  = col_character(),  
      wert          = col_double()  
    )  
  )  
}  
  
v1 <- daten_einlesen("data/lieferung_2024q4.csv")  
v2 <- daten_einlesen("data/lieferung_2025q1.csv")
```

## Praxis

Eine einzige Einlesefunktion bedeutet: identische Spaltentypen, identische Behandlung von Sonderfällen, egal welche Quelldatei. Das ist die Grundlage jedes fairen Vergleichs.

# Den Schlüssel festlegen

Jeder Vergleich braucht einen eindeutigen Schlüssel: die Spalten, die eine Zeile identifizieren. Hier ist das `jahr`, `quartal`, `bereich_code`.

```
schluessel <- c("jahr", "quartal", "bereich_code")  
  
# Prüfen, ob der Schlüssel wirklich eindeutig ist  
v1 %>% count(across(all_of(schluessel))) %>% filter(n > 1)  
# 0 Zeilen = Schlüssel eindeutig
```

## Stolperfalle

Ist der Schlüssel nicht eindeutig, vervielfacht ein Join die Zeilen (kartesisches Produkt). Prüfen Sie das immer zuerst, bevor Sie Differenzen berechnen.

# Beide Stände per Join zusammenführen

Ein `full_join()` behält alle Schlüssel aus beiden Datenständen. Über `suffix` werden die `wert`-Spalten unterscheidbar.

```
vergleich <- full_join(
  v1, v2,
  by = schluessel,
  suffix = c("_v1", "_v2")
) %>%
# bereich_name steht nicht im Schlüssel und wird daher auch
# gesuffixt: zu einer sauberen Spalte zusammenführen
mutate(bereich_name = coalesce(bereich_name_v2, bereich_name_v1)) %>%
select(-bereich_name_v1, -bereich_name_v2)

vergleich %>%
select(jahr, quartal, bereich_code, wert_v1, wert_v2) %>%
head()
```

`full_join()` ist die richtige Wahl, weil ein Schlüssel in genau einem Stand fehlen darf (neue oder weggefallene Zeilen). Diese erscheinen dann als `NA`.

## Stolperfalle

Spalten, die nicht im Schlüssel stehen (hier `bereich_name`), erhalten beim Join ebenfalls den Suffix: aus `bereich_name` werden `bereich_name_v1` und `bereich_name_v2`. `coalesce()` führt sie wieder zu einer Spalte zusammen.

# Absolute und relative Differenz berechnen

```
vergleich <- vergleich %>%
  mutate(
    diff_abs = wert_v2 - wert_v1,
    # gegen NA und Division durch 0 absichern
    diff_rel = if_else(is.na(wert_v1) | wert_v1 == 0,
                      NA_real_, diff_abs / wert_v1),
    status = case_when(
      is.na(wert_v1)           ~ "neu in v2",
      is.na(wert_v2)           ~ "fehlt in v2",
      near(diff_abs, 0)        ~ "unverändert",
      TRUE                     ~ "revidiert"
    )
  )

vergleich %>% count(status)
```

`diff_abs` zeigt die Größe der Änderung in Mio. EUR, `diff_rel` die relative Bedeutung. Beides ist nötig: eine kleine absolute Änderung kann relativ groß sein und umgekehrt.

## Stolperfalle

Zwei Fallstricke bei Fließkommazahlen: `diff_abs / wert_v1` ergibt bei fehlendem oder null Basiswert `NA` bzw. `Inf`, deshalb der `if_else()`-Guard. Und exakte Gleichheit prüfen Sie mit `near(diff_abs, 0)` statt `diff_abs == 0`, weil winzige Rundungsreste sonst als „revidiert“ durchrutschen.

# Einen Schwellenwert für „auffällig“ setzen

Nicht jede Differenz ist berichtenswert. Ein Schwellenwert trennt Rundungsrauschen von echten Revisionen.

```
schwelle_rel <- 0.01    # 1 Prozent relative Abweichung

auffaellig <- vergleich %>%
  filter(status == "revidiert", abs(diff_rel) >= schwelle_rel) %>%
  arrange(desc(abs(diff_rel))) %>%
  select(jahr, quartal, bereich_code, bereich_name,
         wert_v1, wert_v2, diff_abs, diff_rel)

auffaellig
```

## Praxis

Wählen Sie den Schwellenwert fachlich, nicht technisch: Was gilt im Veröffentlichungskontext als revisionsrelevant? Dokumentieren Sie die gewählte Grenze im Report.

# Neue und weggefallene Schlüssel mit anti\_join()

`anti_join(a, b)` liefert die Zeilen aus `a`, deren Schlüssel in `b` nicht vorkommt. Damit isolieren Sie Zu- und Abgänge ohne `NA`-Filterei.

```
# Nur in Vintage 2 (z. B. das neue Quartal 2025Q1)
neu_in_v2 <- anti_join(v2, v1, by = schluessel)

# Nur in Vintage 1 (in v2 weggefallen)
weg_in_v2 <- anti_join(v1, v2, by = schluessel)

neu_in_v2 %>% distinct(jahr, quartal)
# 2025 1
```

`anti_join()` beantwortet die Frage „Was ist hinzugekommen, was ist verschwunden?“ direkt, ohne den vollen Vergleich.

# Vier Bausteine eines vollständigen Vergleichs

## Über Joins

- `full_join()` : alle Schlüssel, Werte nebeneinander
- `anti_join()` (v2, v1): Zugänge
- `anti_join()` (v1, v2): Abgänge
- `mutate()` + `case_when()` : Differenz und Status

## Über Vergleichspakete

- `waldo::compare()` : präziser Objektvergleich, schöne Ausgabe
- `diffdf::diffdf()` : tabellarischer Vergleich mit Schlüssel

Joins liefern den fachlichen Report, die Vergleichspakete liefern die schnelle Kontrolle „Sind diese beiden Objekte identisch?“.

# Objekte vergleichen mit `waldo::compare()`

`waldo::compare()` beschreibt Unterschiede zwischen zwei R-Objekten lesbar: Welche Spalte, welche Position, welcher Wert.

```
library(waldo)

# Identisch? Dann meldet waldo "No differences"
compare(v1, v1)

# Unterschiede zwischen den Ständen
compare(
  arrange(v1, jahr, quartal, bereich_code),
  arrange(v2, jahr, quartal, bereich_code)
)
```

## Stolperfalle

`waldo` vergleicht auch Reihenfolge und Spaltentypen. Sortieren Sie beide Objekte vorher nach dem Schlüssel und stellen Sie gleiche Typen sicher, sonst meldet `waldo` Scheinunterschiede.

# Tabellen vergleichen mit `diffdf::diffdf()`

`diffdf::diffdf()` ist auf Data Frames mit einem Schlüssel zugeschnitten und gibt einen strukturierten Bericht: geänderte Werte, nur links, nur rechts.

```
library(diffdf)

diffdf(
  base      = v1,
  compare   = v2,
  keys      = c("jahr", "quartal", "bereich_code")
)
# Meldet u. a.:
# - Not in COMPARE / Not in BASE (Schlüsselunterschiede)
# - Values mismatch in 'wert'

# Ergebnis als Liste weiterverarbeiten
unterschiede <- diffdf(v1, v2, keys = schluessel,
                      suppress_warnings = TRUE)
```

`diffdf` eignet sich gut als automatische Prüfung: Es nennt Schlüssel- und Wertunterschiede getrennt.

# Zwei Excel-Dateien vergleichen

Dasselbe Muster funktioniert für Excel-Dateien vom Fachbereich. Einlesefunktion, Schlüssel, Join, Differenz.

```
library(readxl)

excel_einlesen <- function(pfad, blatt = "Daten") {
  read_excel(pfad, sheet = blatt, skip = 3) %>%
    rename_with(tolower)
}

daten_a <- excel_einlesen("data/lieferung_a.xlsx")
daten_b <- excel_einlesen("data/lieferung_b.xlsx")

excel_vergleich <- full_join(daten_a, daten_b, by = schluessel,
                             suffix = c("_a", "_b")) %>%
  mutate(diff_abs = wert_b - wert_a,
         diff_rel = if_else(is.na(wert_a) | wert_a == 0,
                             NA_real_, diff_abs / wert_a),
         geaendert = !is.na(diff_abs) & !near(diff_abs, 0))
```

## Praxis

Eine zentrale Einlesefunktion mit festem `skip` / `range` macht den Vergleich robust gegen die typischen Excel-Eigenheiten: Titelzeilen, verschobene Bereiche, Groß- und Kleinschreibung der Spaltennamen.

# Den Abweichungs-Report exportieren

Das Teilergebnis gehört als Datei in die Ablage: prüfbar, datierbar, weitergebbar.

```
library(openxlsx)

# Kompakter CSV-Report (auditierbar, versionierbar)
write_csv(auffaellig, "output/abweichungen_2025q1.csv")

# Mehrblatt-Excel: Auffällige, Zugänge, Abgänge getrennt
write.xlsx(
  list(
    auffaellig = auffaellig,
    neu_in_v2  = neu_in_v2,
    weg_in_v2  = weg_in_v2
  ),
  file = "output/abweichungsreport_2025q1.xlsx"
)
```

Für einen erläuterten Bericht (Text plus Tabellen plus Grafik) rendern Sie ein Quarto-Dokument: ein `.qmd` mit demselben Vergleichscode erzeugt reproduzierbar HTML oder PDF (mehr dazu in Teil VIII).

# Den Vergleich als Funktion kapseln

Wiederholbar wird der Prozess erst als Funktion: zwei Pfade hinein, ein Report heraus. Die Funktion bündelt Einlesen, Join und Differenzberechnung.

```
abweichungsreport <- function(pfad_alt, pfad_neu,
                              schluessel, schwelle_rel = 0.01) {
  alt <- daten_einlesen(pfad_alt)
  neu <- daten_einlesen(pfad_neu)

  full_join(alt, neu, by = schluessel, suffix = c("_alt", "_neu")) %>%
    mutate(
      bereich_name = coalesce(bereich_name_neu, bereich_name_alt),
      diff_abs     = wert_neu - wert_alt,
      diff_rel     = if_else(is.na(wert_alt) | wert_alt == 0,
                             NA_real_, diff_abs / wert_alt),
      auffaellig  = !is.na(diff_rel) & abs(diff_rel) >= schwelle_rel
    ) %>%
    select(-bereich_name_alt, -bereich_name_neu)
}
```

Dieselben Guards wie zuvor ( `if_else()` gegen `NA` /0, `coalesce()` für `bereich_name` ) stecken jetzt einmal in der Funktion und gelten für jeden Aufruf.

# Die Vergleichsfunktion anwenden

Ein Aufruf, zwei Pfade: derselbe geprüfte Code für jedes Paar von Datenständen.

```
report <- abweichungsreport(  
  "data/lieferung_2024q4.csv",  
  "data/lieferung_2025q1.csv",  
  schluessel = c("jahr", "quartal", "bereich_code")  
)  
  
report %>% filter(auffaellig) %>% arrange(desc(abs(diff_rel)))
```

## Praxis

Der Schwellenwert ist ein Parameter mit Vorgabe ( `schwelle_rel = 0.01` ): Standardfall ohne Angabe, im Einzelfall überschreibbar. So bleibt die fachliche Grenze sichtbar und dokumentiert.

# Revisionsanalyse und Vier-Augen-Vergleich

## Praxis

**Revisionsanalyse:** Jeder neue Datenstand revidiert frühere Werte. Der Vergleich zweier Datenstände ( `full_join` plus `diff_abs / diff_rel` ) ist die Revisionsanalyse: Er zeigt pro Bereich und Quartal, wie stark sich der Wert zwischen zwei Ständen geändert hat. Das ist Pflichtdokumentation vor jeder Veröffentlichung.

**Vier-Augen-Vergleich:** Wenn zwei Personen denselben Prozess rechnen oder eine R- gegen eine Excel-Version geprüft wird, ist `waldo::compare()` bzw. `diffdf::diffdf()` der objektive Schiedsrichter: kein Durchscrollen, sondern eine exakte Liste der abweichenden Zellen. Ergebnis und Schwellenwert wandern in den Report, nicht in eine Excel-Zellnotiz.

# Zusammenfassung Teil VII

- Einen Einleseprozess parametrisieren: eine Funktion, ein Dateipfad-Parameter, identische Behandlung aller Quelldateien
- `full_join()` über den Schlüssel, dann `diff_abs` / `diff_rel` und ein fachlicher Schwellenwert für „auffällig“
- `anti_join()` für Zugänge und Abgänge (z. B. neues Quartal 2025Q1)
- `waldo::compare()` und `diffdf::diffdf()` für den schnellen, objektiven Objekt-/Tabellenvergleich
- Report als CSV oder Mehrblatt-Excel exportieren, für Erläuterungen ein Quarto-Dokument

# **Teil VIII: Dokumentation, R-Glossar und Eingabemasken für die Vertretung**

# Was Sie in diesem Teil tun

- Prozesse innerhalb des Codes dokumentieren: Kommentare, roxygen-Stil, Abschnittsmarken
- Prozesse außerhalb des Codes dokumentieren: README und reproduzierbarer Quarto-Bericht
- Ein gemeinsames R-Glossar bzw. eine R-Bibliothek mit VGR-Lösungen aufbauen
- Onboarding und Vertretung vorbereiten: was eine Nachfolge wirklich braucht
- Eine minimale Shiny-Eingabemaske als bedienbares Backup für Laien erstellen

## Praxis

Leitfrage des Teils: Wenn Sie zwei Wochen im Urlaub sind und die Ausgabe trotzdem raus muss, wer kann den Prozess übernehmen und woran orientiert sich diese Person?

# Warum Dokumentation in der amtlichen Statistik zählt

VGR-Prozesse laufen quartalsweise, oft jahrelang, und wechseln im Lauf der Zeit den Bearbeiter. Eine Berechnung ohne Dokumentation ist genau so lange wertvoll, wie die Person verfügbar ist, die sie geschrieben hat.

- Reproduzierbarkeit: dieselbe Auswertung muss jederzeit dasselbe Ergebnis liefern
- Prüfbarkeit: Revisionen und Korrekturen müssen nachvollziehbar bleiben
- Übergabe: Vertretung, Krankheit und Stellennachfolge dürfen den Prozess nicht stoppen

## Praxis

In Excel steckt das Wissen oft in Köpfen, verstreuten Zellkommentaren und gewachsenen Mappen. In R verlagern Sie es in lesbaren Code und versionierte Textdateien: das ist übergebbar.

# Dokumentation IM Code: aussagekräftige Kommentare

Ein Kommentar erklärt das Warum, nicht das Was. Das Was steht bereits im Code.

```
# schwach: wiederholt nur den Code
x <- x * 1.19    # x mal 1.19

# gut: erklärt den fachlichen Grund
# Nettowerte auf Bruttowerte umrechnen (Regelsteuersatz 19 %),
# da die Quelleinheit ab 2024Q3 netto liefert.
wert_brutto <- wert_netto * 1.19
```

## Stolperfalle

Veraltete Kommentare sind schlimmer als keine: Sie führen die Vertretung in die Irre. Pflegen Sie den Kommentar mit, wenn Sie den Code ändern.

# Lesbarer Code ist die beste Doku

Code, der sich selbst erklärt, braucht weniger Kommentare. Sprechende Namen und kleine Schritte ersetzen ganze Erläuterungsabsätze.

```
# schwer lesbar
d <- d[d$j == 2024 & d$q == 4, ]
r <- aggregate(d$w, list(d$b), sum)

# lesbar: Namen und Pipe machen den Ablauf klar
aktuelles_quartal <- daten %>%
  filter(jahr == 2024, quartal == 4)

bws_je_bereich <- aktuelles_quartal %>%
  group_by(bereich_code) %>%
  summarise(wert = sum(wert), .groups = "drop")
```

Gute Namen für Objekte und Funktionen sind die erste Form von Dokumentation.

# Abschnittsmarken: lange Skripte gliedern

RStudio erkennt Kommentarzeilen, die mit `----`, `####` oder `====` enden, als Abschnitt. Sie erscheinen im Gliederungsmenü (Code-Outline) und lassen sich ein- und ausklappen.

```
# 01 Pakete und Pfade -----  
library(tidyverse)  
library(here)  
  
# 02 Daten einlesen -----  
daten <- read_csv(here("data", "lieferung_2024q4.csv"))  
  
# 03 Aufbereitung -----  
# ...  
  
# 04 Ausgabe und Export -----  
# ...
```

Mit `Strg` + `Shift` + `0` öffnen Sie die Gliederung und springen direkt zum gesuchten Abschnitt.

# roxygen-Stil für Funktionen

Funktionen, die Sie weitergeben, dokumentieren Sie mit `#'`-Kommentaren im roxygen-Stil direkt über der Definition. Das ist der R-Standard und lässt sich später automatisch in Hilfeseiten umwandeln.

```
#' BWS-Anteile je Wirtschaftsbereich berechnen  
#'  
#' @param df Tibble mit Spalten jahr, quartal, bereich_code, bereich_name, wert  
#' @param jahr_wahl Jahr, das gefiltert wird (Integer).  
#' @param quartal_wahl Quartal 1 bis 4 (Integer).  
#' @return Tibble je Bereich mit Spalte `anteil` (Summe 1).  
#' @examples  
#' bws_anteile(daten, 2024, 4)  
  
bws_anteile <- function(df, jahr_wahl, quartal_wahl) {  
  df %>%  
    filter(jahr == jahr_wahl, quartal == quartal_wahl) %>%  
    group_by(bereich_code, bereich_name) %>%  
    summarise(wert = sum(wert), .groups = "drop") %>%  
    mutate(anteil = wert / sum(wert))  
}
```

# roxygen lesen und nutzen

## Praxis

Der Block oben sagt der Vertretung in vier Zeilen: was die Funktion tut, was sie erwartet, was sie zurückgibt und wie man sie aufruft. Genau diese Fragen stellt jeder, der eine fremde Funktion zum ersten Mal benutzt.

- `@param` : jedes Argument mit Bedeutung und erwartetem Typ
- `@return` : Form und Inhalt des Ergebnisses
- `@examples` : ein lauffähiger Aufruf als Einstieg

Sobald die Funktionen in einem Paket liegen, erzeugt `devtools::document()` aus diesen Blöcken die `?bws_anteile` -Hilfeseite.

# Dokumentation außerhalb des Codes: README

Eine `README.md` im Projektordner beantwortet die Fragen, die kein einzelnes Skript klärt: Was tut dieses Projekt, wie startet man es, woher kommen die Daten.

```
# VGR-Quartalsdaten

## Zweck
Berechnet die BWS-Anteile je Wirtschaftsbereich aus den
Quartalsdaten und exportiert Tabelle + Grafik.

## So starten Sie den Prozess
1. RStudio-Projekt `projekt.Rproj` öffnen
2. Quartalsdatei nach `data/` legen (Schema siehe unten)
3. `source("R/00_run_all.R")` ausführen
4. Ergebnis liegt in `output/`

## Datenschema
data/lieferung_JJJJqQ.csv: jahr, quartal, bereich_code,
bereich_name, wert (Mio. EUR, jeweilige Preise)
```

Die README ist das Erste, was eine Vertretung öffnet. Halten Sie sie kurz und aktuell.

# Reproduzierbarer Quarto-Bericht

Ein Quarto-Dokument verbindet Erklärtext, Code und Ergebnis in einer Datei. Beim Rendern läuft der Code frisch, Tabellen und Grafiken entstehen neu: der Bericht kann nicht stillschweigend veralten.

```
---  
title: "VGR-Quartalsbericht 2024Q4"  
format: html  
---  
  
Diese Auswertung basiert auf `lieferung_2024q4.csv`.  
  
```${r}  
library(tidyverse)  
daten <- read_csv("data/lieferung_2024q4.csv")  
bws_anteile(daten, 2024, 4)  
```
```

## Praxis

Ein Quarto-Bericht ersetzt das manuelle Zusammenkopieren von Zahlen, Tabellen und Diagrammen aus R nach Word oder Excel. Sie rendern einmal und erhalten ein konsistentes, datierbares Dokument.

# Override- und Audit-Tabelle statt Excel-Zellkommentaren

In Excel landen manuelle Eingriffe in Zellkommentaren oder eingefärbten Feldern: weder durchsuchbar noch reproduzierbar. In R führen Sie stattdessen eine eigene Tabelle der Eingriffe.

```
overrides <- tribble(
  ~bereich_code, ~jahr, ~quartal, ~wert_neu, ~grund, ~bearbeiter, ~datum,
  "GI",          2024, 4,          51200,    "Spätmeldung Quelle", "Devadze", "2025-01-15",
  "K",           2024, 4,          18750,    "Korrektur Tippfehler", "Devadze", "2025-01-16"
)
```

## Praxis

Diese Tabelle ist Dokumentation und Eingriff in einem: Sie wird per Join auf die Daten angewandt (siehe Teil V) und ist zugleich das vollständige, durchsuchbare Protokoll aller Korrekturen.

# Vorteile der Audit-Tabelle

## Excel-Zellkommentar

- an eine Zelle gebunden
- nicht durchsuchbar
- geht beim Umbau verloren
- kein Wer und kein Wann

## R-Override-Tabelle

- eigene Zeilen mit Schlüssel
- filter- und sortierbar
- versioniert in Git
- Grund, Bearbeiter, Datum dabei

Jeder manuelle Eingriff hinterlässt eine nachvollziehbare Spur: das ist in der amtlichen Statistik der Kern einer prüfbaren Revision.

# R-Glossar und R-Bibliothek: die Idee

Jede Fachkraft löst dieselben wiederkehrenden Probleme: Quelldatei einlesen, Gliederung umschlüsseln, Quartale fortschreiben. Ein gemeinsames Glossar bzw. eine gemeinsame Bibliothek sammelt diese Lösungen an einem Ort.

- **Glossar:** kurze Einträge „Problem → Lösung“ mit Code-Schnipsel
- **Bibliothek:** geprüfte Funktionen, die alle im Team aufrufen
- **gemeinsames Repo:** eine Quelle der Wahrheit statt vieler privater Skripte

## Praxis

Statt dass fünf Personen fünf eigene Varianten der Anteilsberechnung pflegen, gibt es eine Funktion `bws_anteile()`, die alle nutzen und gemeinsam verbessern.

# Ein gemeinsames Paket als Bibliothek

Ein R-Paket ist die saubere Form der gemeinsamen Bibliothek. Die Struktur ist schlank und der Einstieg übersichtlich (Skizze der Ordnerstruktur).

```
vgrtools/  
  DESCRIPTION      # Name, Version, Abhängigkeiten  
  NAMESPACE       # was das Paket nach außen anbietet  
  R/  
    einlesen.R      # read_daten()  
    anteile.R       # bws_anteile()  
    reklass.R       # reklassifizieren()  
  man/             # Hilfeseiten (aus roxygen erzeugt)
```

```
# nach Installation aufrufbar:  
library(vgrtools)  
daten <- read_daten("data/lieferung_2024q4.csv")  
bws_anteile(daten, 2024, 4)
```

Funktionen mit roxygen-Blöcken plus `devtools::document()` ergeben automatisch die Hilfe `?`  
`bws_anteile`.

# RStudio-Code-Snippets

Für kurze, immer gleiche Codegerüste eignen sich Snippets: Sie tippen ein Kürzel und Tab, RStudio fügt die Vorlage ein. Definiert unter Tools → Edit Code Snippets.

```
snippet vgrlieferung
  ${1:daten} <- read_csv(
    here("data", "${2:lieferung_2024q4.csv}"),
    col_types = cols(
      jahr = col_integer(),
      quartal = col_integer(),
      wert = col_double()
    )
  )
```

## Stolperfalle

Snippet-Definitionen müssen mit Tabs eingerückt sein, nicht mit Leerzeichen. RStudio meldet sonst einen Fehler beim Speichern.

# Namens- und Stilkonventionen

Eine Bibliothek lebt davon, dass alle nach denselben Regeln schreiben. Halten Sie wenige, klare Konventionen schriftlich fest.

- Objekt- und Funktionsnamen in `snake_case`, deutsch und sprechend
- Funktionen sind Verben ( `reklassifizieren` ), Daten sind Substantive ( `daten` )
- ein einheitlicher Zuweisungsoperator ( `<-` ) und die tidyverse-Pipe ( `%>%` )
- Pfade über `here::here()`, nie absolute Pfade ins Skript schreiben

## Praxis

`styler` formatiert Code automatisch nach Regelwerk, `lintr` prüft auf Abweichungen. So bleibt der Stil im Team einheitlich, ohne dass jemand von Hand nacharbeitet.

# Versionierung mit Git: das Minimum

Git protokolliert jede Änderung am Code und an den Textdateien. Für die Zusammenarbeit am Glossar reichen wenige Befehle.

```
git init                # Repo einmalig anlegen
git add .               # Änderungen vormerken
git commit -m "bws_anteile ergänzt" # Stand sichern
git pull               # Stand der anderen holen
git push               # eigenen Stand teilen
```

## Praxis

Git beantwortet die Revisionsfragen der amtlichen Statistik von selbst: Wer hat was wann geändert und wie sah der Prozess zum Zeitpunkt des letzten Datenstands aus? Mit `git log` und `git blame` ist das jederzeit belegbar.

# Onboarding und Vertretung vorbereiten

Eine Nachfolge übernimmt den Prozess nur dann reibungslos, wenn sie alles an einem Ort findet. Stellen Sie sich vor, Sie sind ab morgen nicht erreichbar.

- **README:** Zweck, Start, Datenschema, Ablage der Ergebnisse
- `00_run_all.R`: ein Skript, das den ganzen Prozess von oben bis unten ausführt
- **Override-/Audit-Tabelle:** alle manuellen Eingriffe nachvollziehbar
- **Glossar/Bibliothek:** die wiederkehrenden Lösungen als geprüfte Funktionen
- **Git-Historie:** der Werdegang des Prozesses

## Praxis

Testfrage für die Übergabe: Kann eine Kollegin allein anhand der README die nächste Quartalsauswertung rechnen, ohne Sie zu fragen? Wenn nein, fehlt Dokumentation.

# Benutzeroberflächen für Laien mit Shiny

Manche Vertretungen kennen R kaum. Für sie baut Shiny eine bedienbare Oberfläche um den dokumentierten Code: Parameter wählen, Knopf drücken, Ergebnis ansehen. Eine Shiny-App besteht immer aus zwei Teilen.

**ui**

das Aussehen: Eingabefelder, Knöpfe, Ausgabebereiche. Was die Nutzerin sieht und anklickt.

**server**

die Logik: liest die Eingaben, rechnet, erzeugt die Ausgabe. Was im Hintergrund passiert.

`shinyApp(ui, server)` verbindet beide Teile zur lauffähigen App.

# Reaktivität verstehen

Shiny ist reaktiv: Ändert sich eine Eingabe, rechnet der zugehörige Ausgabeblock automatisch neu. Sie beschreiben die Abhängigkeiten, nicht den Ablauf.

```
# input$jahr ist eine Eingabe aus dem ui
# output$tabelle ist ein Ausgabebereich im ui

output$tabelle <- renderTable({
  bws_anteile(daten, input$jahr, input$quartal)
})
```

Sobald die Nutzerin `jahr` ändert, läuft der Block in `renderTable()` von selbst erneut. `eventReactive()` koppelt die Berechnung stattdessen an einen Knopfdruck.

# Eine minimale Eingabemaske

Diese App wählt Jahr und Quartal, rechnet auf Knopfdruck die BWS-Anteile und zeigt das Ergebnis als Tabelle und Grafik mit CSV-Download.

```
library(shiny)
library(tidyverse)

ui <- fluidPage(
  titlePanel("VGR-Anteile je Bereich"),
  sidebarLayout(
    sidebarPanel(
      numericInput("jahr", "Jahr", value = 2024, min = 2018, max = 2025),
      numericInput("quartal", "Quartal", value = 4, min = 1, max = 4),
      actionButton("rechnen", "Berechnen")
    ),
    mainPanel(
      tableOutput("tabelle"),
      plotOutput("plot"),
      downloadButton("export", "Als CSV speichern")
    )
  )
)
```

Das `sidebarLayout` stellt die Eingaben links neben das Ergebnis rechts.

# Server, Grafik und Download

```
server <- function(input, output, session) {  
  ergebnis <- eventReactive(input$rechnen, {  
    bws_anteile(daten, input$jahr, input$quartal)  
  })  
  
  output$tabelle <- renderTable(ergebnis())  
  
  output$plot <- renderPlot({  
    ggplot(ergebnis(), aes(bereich_code, anteil)) +  
      geom_col()  
  })  
  
  output$export <- downloadHandler(  
    filename = function() paste0("anteile_", input$jahr, "q", input$quartal, ".csv"),  
    content = function(file) write_csv(ergebnis(), file)  
  )  
}
```

`eventReactive()` rechnet erst beim Klick; Tabelle, Grafik und Download greifen alle auf dasselbe Ergebnis zu.

# Shiny ergänzt den Code, ersetzt ihn nicht

## Stolperfalle

Die Shiny-App ist nur eine Bedienoberfläche für die bereits geprüften Funktionen wie `bws_anteile()`. Sie ersetzt nicht den dokumentierten Code, sondern macht ihn für Laien bedienbar. Liegt die fachliche Logik in der App selbst statt in getesteten Funktionen, ist sie genauso schwer zu übergeben wie eine undokumentierte Excel-Mappe.

Bauen Sie die App also immer auf Ihrer Bibliothek auf: die App liefert die Knöpfe, das Paket die Rechenlogik.

# Backup-Form für die Vertretung

## Praxis

So fügt sich Teil VIII zusammen: Die dokumentierten Funktionen liegen im gemeinsamen Paket, die manuellen Eingriffe in der Audit-Tabelle, der Ablauf in README und Quarto-Bericht. Die Shiny-Maske setzt obendrauf und erlaubt der Vertretung, eine Standardauswertung zu fahren, auch ohne R-Kenntnisse: Jahr und Quartal wählen, rechnen, exportieren.

Für komplexe Fälle bleibt der Code die Grundlage; für den Routinefall ist die Eingabemaske das verlässliche Backup.

# Zusammenfassung Teil VIII

- Dokumentieren Sie im Code (Kommentare zum Warum, roxygen, Abschnittsmarken) und außerhalb (README, Quarto-Bericht, Audit-Tabelle)
- Bündeln Sie wiederkehrende VGR-Lösungen in einem gemeinsamen Glossar bzw. Paket, mit Konventionen, Snippets und Git
- Eine Übergabe gelingt, wenn README, `00_run_all.R`, Audit-Tabelle, Bibliothek und Git-Historie zusammen den ganzen Prozess erklären
- Shiny macht den dokumentierten Code für Laien bedienbar, ersetzt ihn aber nicht

# Abschluss und nächste Schritte

Sie haben einen vollständigen R-Workflow vom Einlesen bis zur dokumentierten, vertretbaren Produktion gesehen. Bauen Sie das gemeinsame R-Glossar weiter aus und überführen Sie einen echten Excel-Prozess Schritt für Schritt nach R.